

Rafik Hariri University

College of Arts and Sciences

AI-Powered Android App for Job Interview Training with Live Video and CV Analysis

Final Year Project Report

Done By

RAZAN HASBINI

MOHAMAD IBRAHIM

Submitted to

DR. ALI RAMMAL

This project, submitted as part of the requirements for a Bachelor Science degree in Computer Science from the College of Arts and Sciences at Rafik Hariri University is being presented as a senior project.

MECHREF, LEBANON

December 2025

Copyright©2025, all rights reserved

Mohamad ibrahim (2021-0673)

Razan hasbini (2023-0226)

Aknowledgement

We would like to express our sincere gratitude to our supervisor, Dr. Ali Rammal, for always supporting us and believing in our potential. We appreciate his constructive feedback, and valuable support throughout the development of this project. His expertise and vision truly contributed to shaping the quality and success of this project.

We also extend our appreciation to Dr. Mohammad El Abed for his insightful academic contributions and for fostering a supportive learning environment throughout our studies, as well as to Dr. Lara Abou Orm, Professor and Chair of the Computer and Information Systems Department, and Dr .Roa Al Solh.

Finally, we acknowledge the Computer Science Department at Rafik Hariri University provides the resources, facilities, and academic foundation essential to the successful completion of this Final Year Project.

Abstract

strong interview performance and a well-crafted CV play a decisive role in shaping a candidate's employment prospects. However, many students and recent graduates often feel unprepared because they lack real interview exposure, confidence, and clear guidance on how to improve their resumes. This report presents the development of an intelligent Android application designed to address these challenges by offering three integrated capabilities: simulated interview training, automated CV evaluation, and a social space where users can exchange experiences and insights. The mobile platform enables individuals to upload their CVs for structured analysis powered by linguistic and semantic models, while also engaging in interactive practice interviews that generate AI driven questions tailored to each user's background. In addition to analyzing verbal responses, the application leverages on-device MediaPipe models to interpret non-verbal behaviors, such as eye movement, gaze stability, and posture providing a more complete understanding of user performance during an interview session.

Supporting this system is a Go-based backend developed with a layered architectural approach, incorporating handlers, services, repositories, and middleware to manage authentication, profiles, content sharing, notifications, and interview data. The server uses PostgreSQL for relational storage and relies on JWT for secure user session management. On the client side, the Android application follows the MVVM pattern and is implemented with XML layouts, fragments, ViewModels, Retrofit, and OkHttp, while EncryptedSharedPreferences ensures the safe handling of access tokens. Supabase is used for storing media assets such as user and post images, and the entire system is deployed at <https://androidfyp.onrender.com>

Table of Contents

Acknowledgment.....	3
---------------------	---

Abstract	
.....	4 Chapter
1: Introduction and Motivation.....	8
1.2 Aim and Contribution of the Project.....	8
1.3 Report Structure.....	9
Chapter 2: Literature Review.....	11
2.1 Introduction.....	11
2.2 Competitors Review	11
2.3 Platform	12
2.4 Comparison Between Competitors and Our Platform.....	13
2.5 Summary.....	14
Chapter 3: System Requirements and Design.....	14
3.1.1 Functional Requirements	14
3.1.2 Non-Functional Requirements	14
3.1.3 High-Level Architecture	15
3.1.4 Backend Layered Architecture	15
3.1.5 Use Case Diagram	23
3.1.6 System Architecture Diagram.....	24
3.1.7 Sequence Diagram.....	27
3.1.8 Class/Layer Diagram.....	32
Chapter 4: Machine Learning Design.....	30

Chapter 5: System Implementation	33
5.1 Android Application	33
5.1.1 Authorization	34
5.1.2 UI Architecture and MVVM Structure	37
5.1.3 CameraX and ML Integration	39
5.1.4 Speech Recognition and Text to Speech Integration... ..	40
5.1.5 Interview Session Control Flow	41
5.1.6 CV Upload... ..	42
5.1.7 User Profile And Score Retrieval	43
5.2 AI logic Code	44
5.2.1 CV Analysis Pipeline	44
5.2.2 Live Interview Intelligence Engine	45
5.2.3 Feedback And Score Computation... ..	46
5.4 Routing And Endpoints	50
5.5 Application Pages	51
Chapter 6: Survey and Evaluation	61
Chapter 7: Conclusion and Future Work	68
References	69
Appendix A: Database Tables	69

List of Figures

Figure 1 General Comparison of Wazzafak and its competitors.....	14
Figure 2 Backend Layered Flow	22
Figure 3 Use Case Diagram	23
Figure 4 System Architecture Diagram.....	25
Figure 5 Sequence Diagram 1	27
Figure 6 Sequence Diagram 2.....	29
Figure 7 UML Class Diagram.....	32
Figure 8 Machine Learning Workflow.....	32
Figure 9: Sentiment analysis model	34
Figure 10 Android Application Architecture.....	35
Figure 11 Sign up verification email.....	37
Figure 12 Reset Password verification email.....	37
Figure 13 Communication of MVVM	38
Figure 14 REST API endpoints	40
Figure 15 Interview flow.....	42
Figure 16 CV analysis pipeline.....	45
Figure 17 Splash/Welcome Screen.....	52
Figure 18 Login page	53
Figure 19 Sign Up Page... ..	53
Figure 20 Field Choices	54
Figure 21 Domain choices Example.....	54
Figure 22 Forgot password.....	55
Figure 23 Home Page.....	56
Figure 24 Comments on Posts.....	56
Figure 25 Creating a Post.....	57
Figure 26 Post Success.....	58
Figure 27 Live Interview Page.....	58
Figure 28 Interview Snippet.....	59
Figure 29 Notifications Scre.	60
Figure 30 CV Feedback on CV Screen... ..	60
Figure 31 Profile Screen.....	61
Figure 32 Bottom Nav Bar.....	61

Chapter 1: Introduction and Motivation

1.1 Introduction

In today's competitive job market, securing employment requires more than technical qualifications. Recruiters increasingly evaluate candidates based on communication skills, confidence, clarity of expression, and overall professional presentation during interviews. For many students and fresh graduates, the job interview represents a significant challenge due to limited exposure, lack of structured preparation, and minimal real world practice.

Similarly, the curriculum vitae, widely known as the CV, remains a critical component of the recruitment process. As the first document reviewed by employers, a poorly structured or incomplete CV can significantly reduce a candidate's chances of advancing to the interview stage. Many students struggle to assess the quality of their CV or identify areas that require improvement, resulting in missed opportunities.

The emergence of artificial intelligence (AI) and mobile technologies presents an opportunity to address these issues by providing accessible, automated, and personalized career preparation tools. This project leverages these technologies to design and develop a platform that supports interview readiness, CV enhancement, and peer engagement through an integrated mobile application.

1.2 Aim and Contribution of the Project

The aim of this project is to develop a comprehensive Android based system, supported by a Go backend, that assists users, especially students and fresh graduates, in preparing for job interviews and improving their CVs through AI-powered analysis and interactive features. The system is designed to offer a realistic training environment, automated feedback mechanisms, and a social space for sharing professional experiences.

The key contributions of this project include:

1. An AI-Enhanced Interview Training Platform

A module that allows users to conduct mock interviews using their mobile device, enabling automated evaluation of performance through speech and behavioral analysis.

2. Intelligent CV Evaluation

An AI-based CV analysis feature that identifies strengths and weaknesses in the user's CV and generates personalized recommendations for improvement.

3. A Social Interaction Environment

A built in community platform that enables users to share posts, interact through likes and comments, and learn from one another's professional experiences.

4. A Secure and Scalable System Architecture

The system incorporates a clean and modular design using MVVM on Android and a layered architecture on the backend, ensuring maintainability, security, and scalability.

5. A Foundation for Future Expansion

The system is intentionally designed for future enhancements such as real time chat communication and job opportunity listings, making it adaptable to evolving user needs.

1.3 Report Structure

This report is organized into seven chapters, each addressing a key aspect of the system's development.

Chapter 2 reviews existing platforms related to interview training, CV evaluation, and social interaction. It compares their features with those of our system and highlights the specific contributions of the proposed platform.

Chapter 3 presents the system requirements and design. It outlines the functional and non-functional requirements, the technologies used, and the overall architecture of both the Android application and the backend system.

Chapter 4 provides an overview of the machine learning components used in the platform, describing the general design approach and the role of AI in interview analysis and CV evaluation.

Chapter 5 explains the implementation of the system, including the Android application structure, backend API development, database integration, and the main features available to users.

Chapter 6 discusses the evaluation of the system, summarizing user feedback and examining the platform's usability and effectiveness.

Chapter 7 concludes the report and outlines potential future enhancements, such as adding job opportunity listings and implementing a real-time chat system.

Chapter 2: Literature Review

2.1 Introduction

In this chapter we go over the results of the research we implemented before starting this project ,including applications that span the same category as Wazzafak , market analysis , and platforms that offer similar services to our app's services.

This analysis was conducted as a first step in guaranteeing people's needs for an application like Wazzafak in the market.

2.2 Competitors Review

Interview Practice Tools

Throughout the process of exploring our competitors ,we came across a lot of websites, apps and even agents that offer analysis and feedback services, and even though the range of such applications might seem small , or limited , each one of these platforms tends to dive into a different path than all of the others.

For instance, Yoodli, one of the most famous websites for analyzing public speaking skills , offers some of our features by giving analysis to videos , and tailored feedback about one's negative points , yet it does not generate the interview environment , and it is not specifically tailored for job interviews either , it includes presentations and even judgement for offer pitching skills , which shifts its direction and highlights wazzafak's ownership of the idea.

Another well-known communication coach is Big interview, this AI is generally used by university students , whom to use it have to record a video of themselves and send it to the AI chat , exactly like Yoodli, and given the fact that wazzafak does all of this with only the press of a button, and users don't have to record themselves or type anything in the chat , it was clear that wazzafak is ought to be needed.

Up next , we have Pramp, which is also widely used .Pramp stands out by not using any AI to assess skills , but instead it connects 2 random users from around the world, and they have to interview each other and assess each other. This idea attracted a big audience

, yet ,even though it is creative ,random people are not as trusted of a source for constructive criticism as much as a well trained AI.

CV Analysis Tools

In the interview practice tools section

For this section ,we will only review the cv analysis tools people use , to learn how much if an addition the feature would be to our application.

The two most famous CV analyzers are Jobscan and ResumeWorded.

Jobscan helps you match your CV/resume against specific job descriptions: it checks keywords, formatting and ATS compliance to improve the chances your CV gets selected by applicant-tracking systems.

This shows that CV analysis is an essential feature for Wazzafak, because platforms like Jobscan and ResumeWorded prove how valuable it is to understand a candidate's strengths, weaknesses, and ATS-readiness before any interview begins. While we cannot compare ourselves directly to these tools , since they only analyze CVs and we offer an entire interview training ecosystem with anxiety detection, AI questioning, and social learning—we can still learn from them. Their popularity reinforces that strong CV analysis is genuinely useful, and it validates our decision to integrate it as part of a larger, more complete experience. From our point of view, it confirms that analyzing a CV early in the process helps us ask better, more personalized interview questions and ultimately provides users with a much deeper and more effective training journey.

2.3 Platform

Unlike single-purpose tools, Wazzafak brings all three into one accessible environment for students and fresh graduates.

Wazzafak also stands out by being a mobile app, not a website or desktop app, which means it is a moving training app that is always in one's pocket, users can train anywhere and everywhere with only the press of a button, without having to take videos of them selves and search web engines for the website and all that. This also means that they also won't have to get up and prepare their environment for long, all they have to do is set their phone in front of them wherever they are.

Thus, it is a low effort practice , with a highly positive effect , because when practice does not require much effort , users tend to do it more often , and one time after the other , they are improving their skills without even noticing, and achieving the objective of Wazzafak .

2.4 Comparison Between Competitors and Our Platform

Feature	Competitor Tools	Our Platform
Interview Practice	Limited to text-based or basic video feedback	AI-powered analysis with behavior & speech evaluation
CV Evaluation	Mostly paid services or limited free tools	Integrated, automated CV scoring within the app
Social Interaction	Only LinkedIn-style professional networking	A closed, training-focused community
Accessibility	Web-heavy, not mobile-first	Fully Android-based, optimized for students

Personalization	Generic feedback	AI-driven personalized recommendations
-----------------	------------------	--

9Figure 1: General Comparison of Wazzafak and its competitors.

2.5 Summary

The proposed system combines elements from interview practice, CV analysis, and social interaction. Unlike existing single-purpose tools, it aims to provide an integrated experience for students and fresh graduates, with clear paths for AI expansion and realtime social engagement.

Chapter 3: System Requirements and Design

Wazzafak AI is designed to represent a platform with a perfect balance between smart dynamic support and social networking support, and in order to achieve that , the following functional and non-functional requirements should be well attained:

3.1 Functional Requirements

3.1.1 User Management

- Users can register using email and password.
- Email of registration should be a valid email and will be verified upon the creation of a new account.
- Users can securely log in and receive JWT tokens for subsequent requests.
- Users can reset or change their passwords.
- Users can view and edit their profile information including adding a profile picture.

3.1.2 Social interactions

- Users can create, edit, or delete posts containing texts and images.
- Users can like and comment on posts and remove them when wanted.

- Users can follow and unfollow other users
- A personalized feed displays accounts of other users in case the user is interested in following them or their posts.
- Notifications are generated for likes , comments, and follows.

3.1.3 Cv Analysis And Interview Preperation

- Users can upload CVs in DOCX, PDF , or TXT format.
- The backend extracts and processes CV text using docconv library.
- The system generated embeddings using Open Ai's textembedding-3 small model.
- Ai performs a structural CV review to identify strengths , weaknesses , and relevant domains.
- Users receive structured CV analysis and personalized feedback.

3.1.4 AI Driven Interview System:

- Users can initiate a live interview session through websocket communication.
- AI Interview questions are generated dynamically using GPT-4o mini.
- Users respond via voice input while the app converts speech to text and transmits it to the backend.
- GPT-4o- evaluates user responses based on clarity, correctness, communication, and confidence.
- Face movement, poses, gaze , eye contact, and hand movement , which are also described as behavioral activity are detected on-device using MediaPipe.

-
- The backend merges linguistic, and behavioral indicators to produce the final feedback report.

Interview history, evaluations, and feedback of the last trial are stored for future review and could be posted for other users to view.

3.2 Non-Functional Requirements

3.2.1 Security

- All sensitive operations require a valid JWT token
- Passwords are hashed using a secure, modern hashing algorithm.
- Refresh tokens and JWTs are stored in Android's EncryptedSharedPreferences.
- Cross-origin requests, rate limiting, and request validation protect the backend.
- CV files and user generated content are uploaded and stored securely.

3.2.2 Performance&Scalability

- The backend must handle real time WebSocket connections with minimum latency.
- AI calls (Open AI API calls) should be optimized to avoid delays during interviews.
- The Android app should perform efficiently on mid-range devices , including Media-pipe inference.

-
- Database queries should use indexed columns and optimized schemas.

3.2.3 Reliability

Temporary fails, such as network loss , or open AI timeouts should not crash interview sessions

- Local Caching (Room Database) should enable the app to remain partially function online.

3.2.4 Usability

- The interface must remain intuitive for users with non-technical backgrounds.
- Navigation should use modern and well known design patterns : MVVM , ViewModel State management, and clear UI segmentation.

3.2.5 Maintainability

- Both Android , and Go backend follow modular , layered architectures.
- Code components must be isolated in 4 different layers: networking , business logic , data access , and UI.
- Any future extensions of AI logic, must be facilitated by clear naming conventions, and documentation.

•

3.3 High-Level Architecture

Wazzafak AI adopts a client-server architecture enhanced with cloud AI services , and device-side Machine Learning. The system contains 4 main layers:

1. Android Client Application (Frontend):

This client functions as the user interface and local pressing unit, and it handles:

- UI rendering (Fragments + Activities with XML layouts)
- Speech to text and real-time camera processing
- MediaPipe Behavioral Tracking (body structure , eye contact , face and brow movement and hand movement)
- Token Storage And Session management
- CV selection and File upload
- Social interactions (following , posting , liking , and commenting)
- Real-time interview WebSocket communication

Technologies used:

- Kotlin
- MVVM architecture
- Retrofit + OkHttp
- Ktor WebSocket client
- CameraX + MediaPipe
- EncryptedSharedPreferences
- Room Database

2. Go Backend API:

The Go backend is the Rule Engine which handles all logic and decisions in the app , such as authentication , AI integration , and data persistence. It also holds the following responsibilities:

- Issuing authentication and JWT tokens • CV parsing, chunking, and embedding generation.
- Interview orchestration through WebSocket.
- Requesting AI responses and evaluations from GPT models.
- Storing session history, evaluations, and behavioral metrics.

- Managing social interaction tools.

Technologies used:

- Golang
- Chi Router
- Postgre SQL (Supabase)
- Pgx driver
- Go -openai client
- WebSocket (gorilla/websocket)

3. Machine Learning Layer:

This layer combines server-side and device-side ML:

Sever side ML:

- **OpenAI models:**
- Text-embedding-3-small for embedding Cvs. Gpt-4o-mini for interview questions and cv-analysis Gpt-4o for answers evaluation and scoring.
- NLP based CV analysis
- Semantic similarity and question generation.
- Multi-criteria answer scoring.

Device-side ML:

- MediaPipe Face Detection
- MediaPipe pose and tracking
- Behavioral signal extraction (gaze shifts, posture instability, hand fidgeting ,brow clutching)

This split structure ensures a smooth combination of both linguistic and habitual analysis, resulting in a high quality platform.

4. Supabase cloud Database:

Supabase is a postgres based database that is hosted on the cloud , and in our project it stores:

- User accounts and social graphs
- CV documents and embeddings
- Evaluations and interview feedback
- Behavioral metrics
- Posts, comments, likes, and follows
- Practice session history

The PgVector extension enables high-dimensional embedding storage.

3.4 Backend Layered Architecture

The backend follows the flow:
HTTP Request → Handler → Service → Repository → Database → Repository → Service → Handler → HTTP Response.

Wzzafak AI Backend Layered Flow

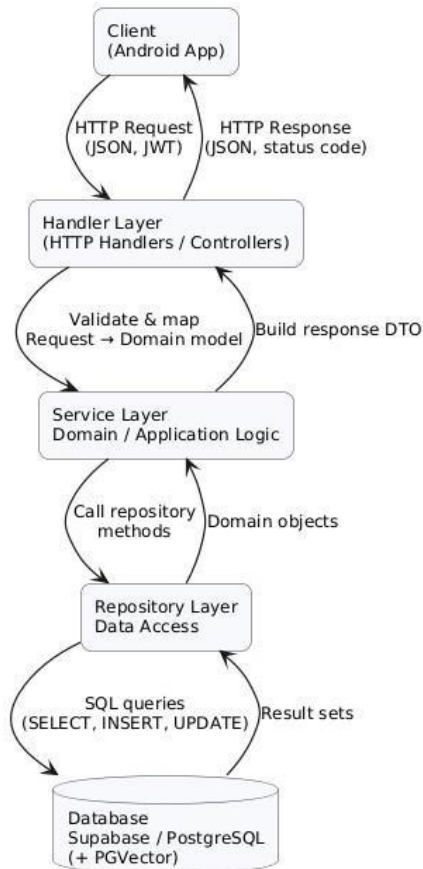


Figure2:Backend layered Flow

The backend follows a multilayered architecture designed to separate concerns and ensure maintainability. Each incoming HTTP request moves through a set of well-defined layers, where controllers validate and translate the request, services execute domain logic, repositories handle data persistence, and the database stores all structured data including CV documents, embeddings, and evaluation results

The main project structure is: myproject/ internal/
 handler/ // handles HTTP
 requests service/ // logic of the app repository/ // Database access model/
 // Data models

middleware/ // Token authentication and logging in database/ // Database connection
 and migrations config/ // Environment and configuration loading utils/ //
 Helper functions main.go // Application entry point

3.5 Use Case Diagram

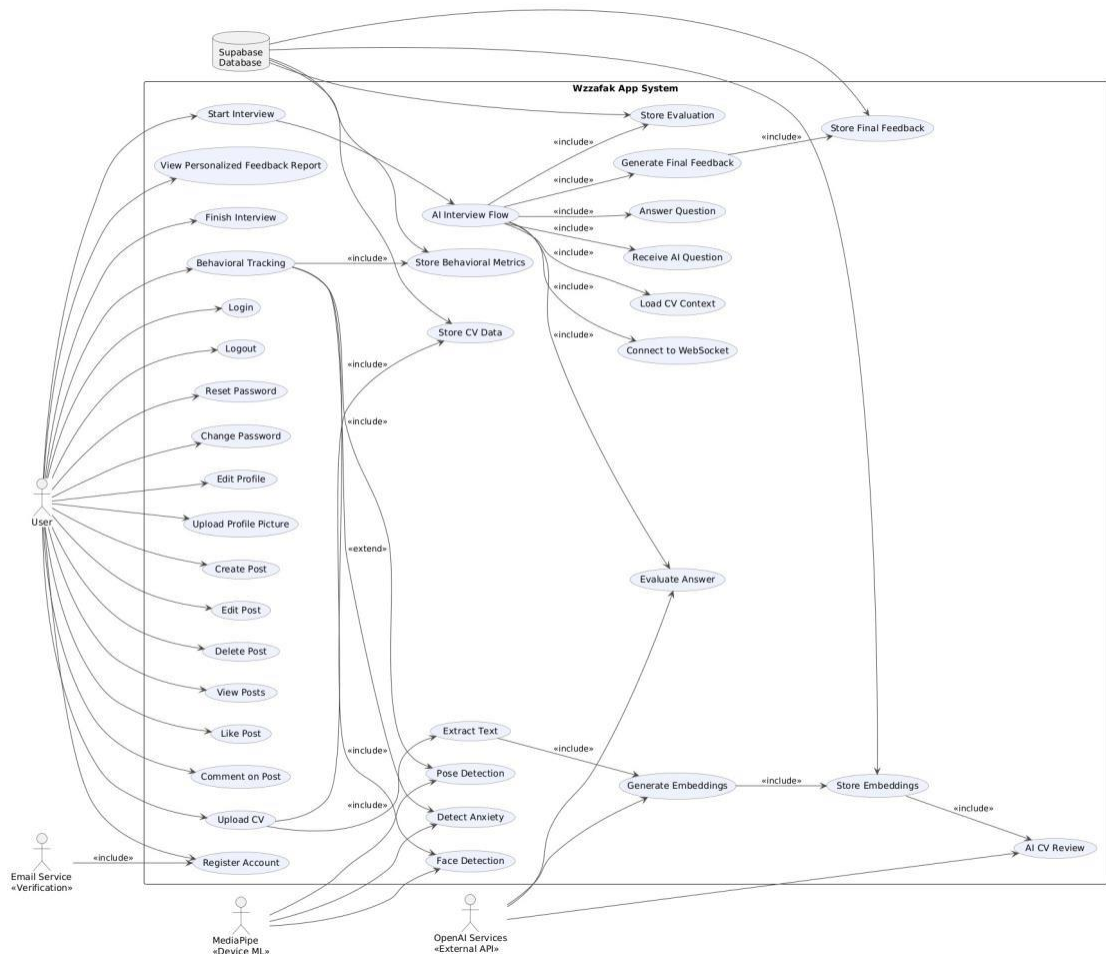


Figure3 : Use Case Diagram

The use case diagram drawn in Figure , shows how all the functionalities the user can do are supported by either external systems or internal components.

It highlights (by keeping them out of the box) the 4 main factors supporting the flow of the features for the user , which are OpenAI Services, MediaPipe, and the Supabase Database, as well as the auxiliary Email Verification Service. The user has the ability to trigger all functionalities>

With the wide range of interactions a user can create , we can assign a simple list which states that a user can log in , manage their profile , change their password, upload a profile picture and edit all these information. Further more , the user can view , post and delete, posts and add likes and comments on either his posts , or other's posts.

The user also utilizes the system's core AI capabilities when they upload their cv , because it initiates a sequence of operations including : extracting text , generating embeddings, storing CV data , or performing an AI-driven CV review through OpenAI services.

As for the interview training , the user begins an interview session which relies on multiple included use cases such as connecting Websocket server , loading CV context , receiving questions, submitting answers , and receiving structured evaluations .

Together, these interactions depict a comprehensive ecosystem, where user authentication, social engagement, CV analysis , AI interview flow and Behavioral assessment interact cohesively. The include and extend relationships clarify which processes are mandatory components of larger workflows in the app.

3.6 System Architecture

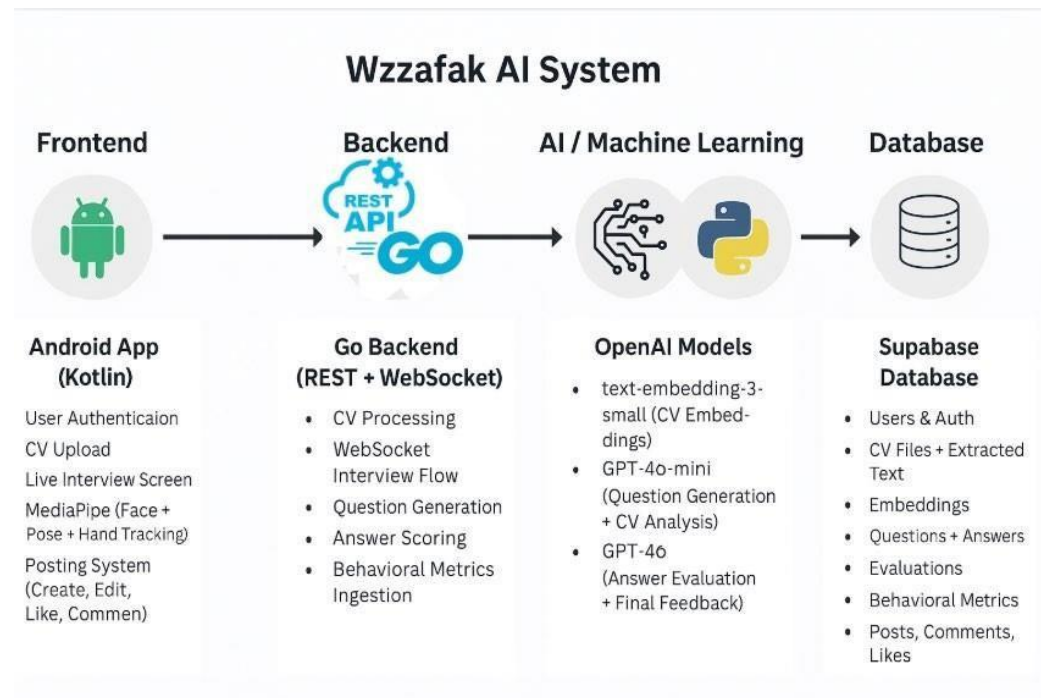


Figure 4: System Architecture Diagram

The system Architecture Diagram represented in Figure , shows how we structured the application after we decided that the intelligence should be distributed across the entire application , including android client, the Go server , and all the machine learning components we integrated .

The system has two primary access points, the client application responsible for handling user interactions along with collecting behavioral signals , and the backend orchestrator managing CV processing , parsing , embedding chunks , plus dealing with interview flows and AI model execution.

However ,on the server side , the central orchestrator is the backend Go server of the ML pipeline, it processes uploaded CVs by extracting tests , generating embeddings , and storing the data properly in Supabase cloud cPostgre database.

During interviews, the orchestrator coordinates with multiple AI models : GPT-4omini for generating context-aware interview questions, GPT-4o for evaluating user answers, and text-embedding-3-small for semantic encoding of CV content. Collectively, these models act in unity as prediction engines , calculating score, feedback and follow up questions based on user input, Performance monitors which are implemented through logging and stored evaluation data, track how well the models perform in realistic sessions, enabling continuous improvement of prompts and evaluation logic.

In parallel, the behavioral signals produced by on-device MediaPipe models , compliment the language based evaluation forming a hybrid assessment pipeline.

All outputs generated throughout this process, such as CV embeddings, questionanswer records, behavioral indicators, and overall performance summaries, are stored in the Supabase database, which acts as the central repository for user activity and interview session history. The system operates through a coordinated workflow in which the Android client gathers user input, while the backend carries out the heavier analytical tasks using the integrated machine learning models. Once the server processes the linguistic, contextual, and behavioral data, it compiles the results into a structured feedback

report and sends it back to the user. By combining CV semantics, real-time interview responses, and observable behavioral cues into a single pipeline, the architecture supports a more realistic simulation of an interview environment and enables deeper insight into user performance without relying on repetitive or isolated ML components.

3.7 Sequence Diagram

In this section, we will be showing two sequence diagrams explaining all the interactions that flow through the app within' every step to perform an action.

There will be two diagrams instead of one, due to the fact that this project includes two different flows and products integrated in one, and so, showing two diagrams instead of a messy one , shows a clearer picture of how the components interact within' this application.

Sequence Diagram 1:

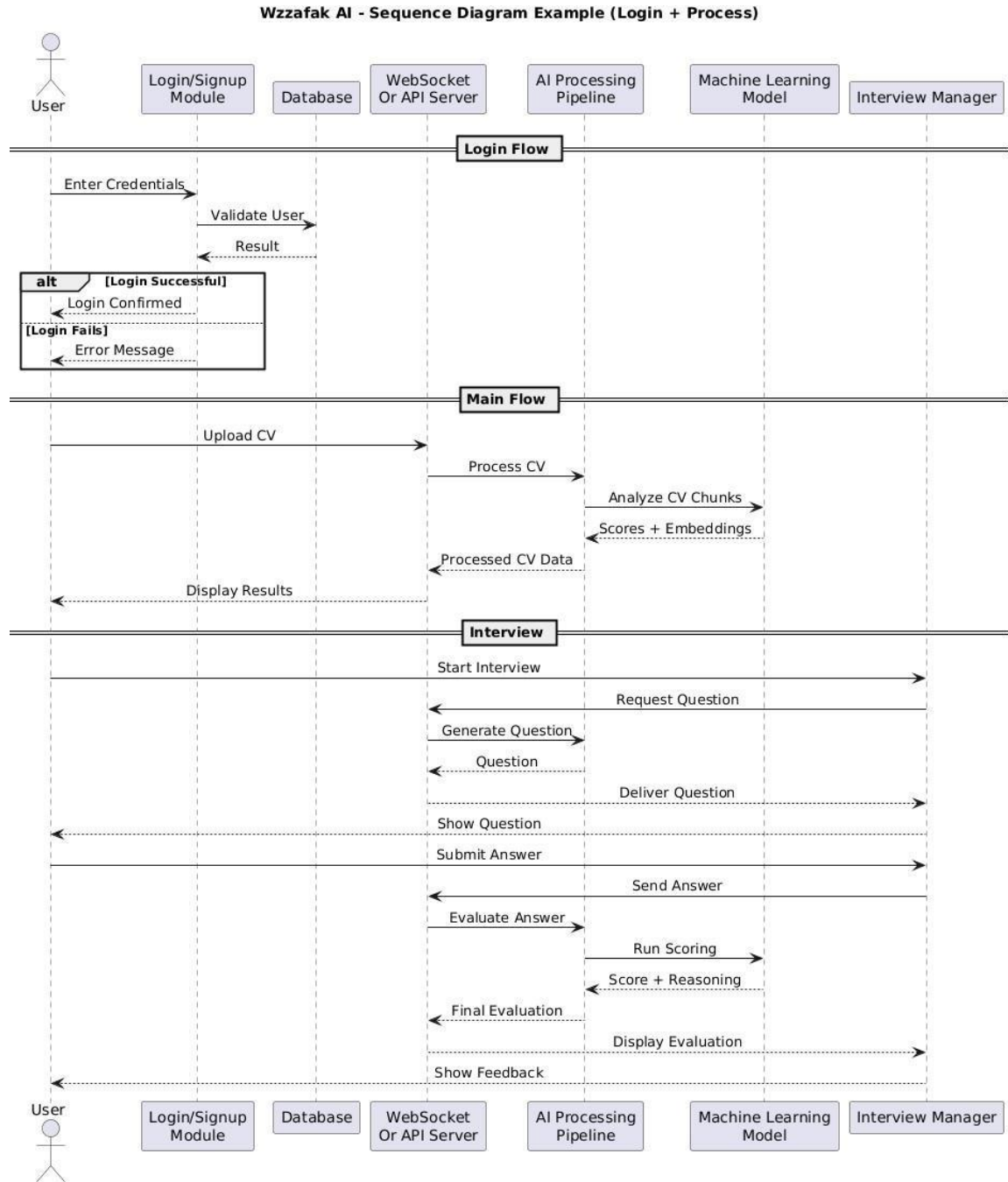


Figure 5: Sequence Diagram (Login + Process)

The Sequence Diagram shown in figure, illustrates the complete workflow of Wazzafak AI system from user authentication to the CV analysis and AI interview sections. It demonstrates how the Android client, backend server, and AI components communicate to deliver a seamless interview experience.

The process begins when the user attempts to log in by entering their credentials into the mobile application. These credentials are then forwarded to the authentication module which verifies them by checking the database components. If the credentials entered exist, the user gains access to the system, and is forwarded to the dashboard, if they don't, the user receives an error notification indicating that the process has failed.

Once authenticated, the user can upload their CV through the app. This file is sent to the backend server which initiates the AI processing pipeline. The pipeline extracts the textual content of the CV, cleans it, and divides it into semantically meaningful chunks. These chunks are then transformed into vector embeddings using the machine learning model, allowing the system to analyze the CV's structure, content, and relevance.

After processing, the system sends a structured summary of the CV back to the client. This enables the app to display an AI enhanced preview of the user's experience

, which later serves as contextual input for generating personalized interview questions.

The final phase of the diagram represents the interactive interview flow. When the user starts the interview, the request is handled by the interview manager, which communicates with the backend to obtain the first question. The backend uses both the processed CV data and the AI reasoning model to generate a tailored interview question. This question is returned through the interview manager and displayed to the user. After the user submits an answer, it is sent back to the API server, which forwards it to the AI pipeline for evaluation. The machine learning model analyzes the response, generates a score, and provides reasoning to support the evaluation. This feedback is then returned to the application and presented to the user. The cycle of question generation, answering, and evaluation continues until the interview session concludes.

Sequence Diagram 2:

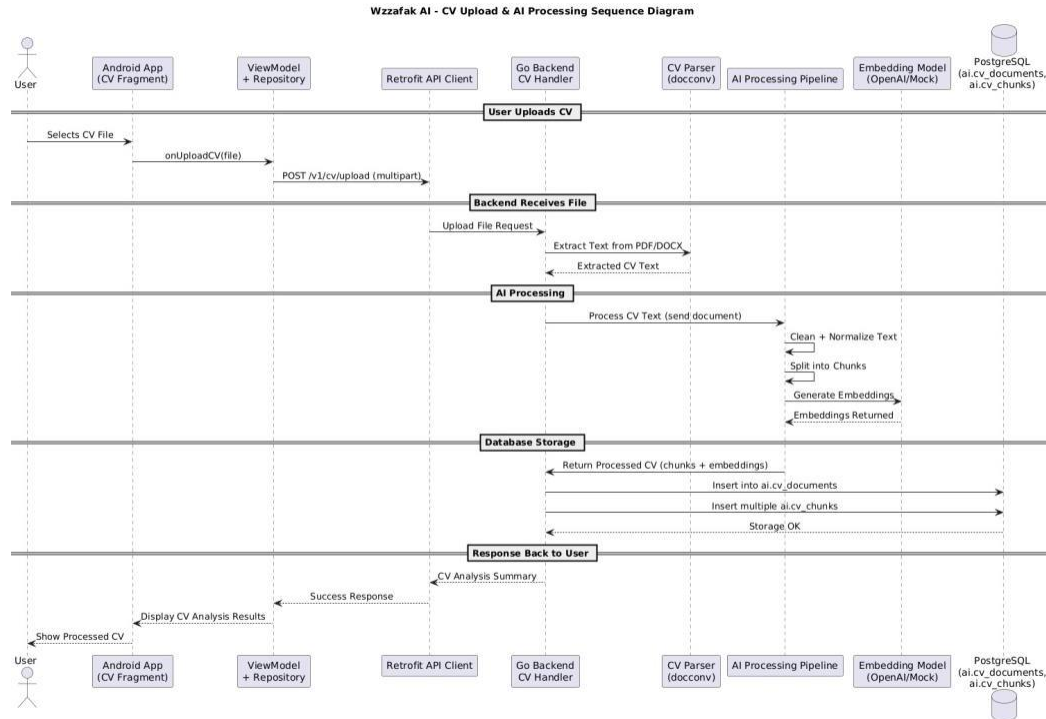


Figure 6: CV Upload & AI processing Sequence Diagram:

The sequence diagram shown in Figure demonstrates the complete workflow of the entire CV parsing and analyzing process. When the user chooses a CV file in the CV Fragment, the android app forwards this file to the ViewModel, which then delegates the upload task to the repository layer. Then , through the Retrofit API client, a multipart POST request is sent to the backend's CV handler. Once the backend receives this file , it triggers the CV Parsing component , which extracts and cleans the contextual content from formats such as PDF or DOCX.

After extracting the text , the backend passes the document or file to the AI processing pipeline where the text is normalized , split into chunks , and then converted into vector embeddings using the selected embedding model.

When the AI pipeline finishes processing , it returns the enriched CV data , which includes , raw text, chunked segments, and embeddings. This information is the inserted into the ai.cv_documents table and its associated ai.cv_chunks entries . Once the data is successfully stored, the backend sends a summary response back through the Retrofit client

to the ViewModel, which updates the UI with the processed CV information that the user can see.

3.8 Class/Layer Diagram

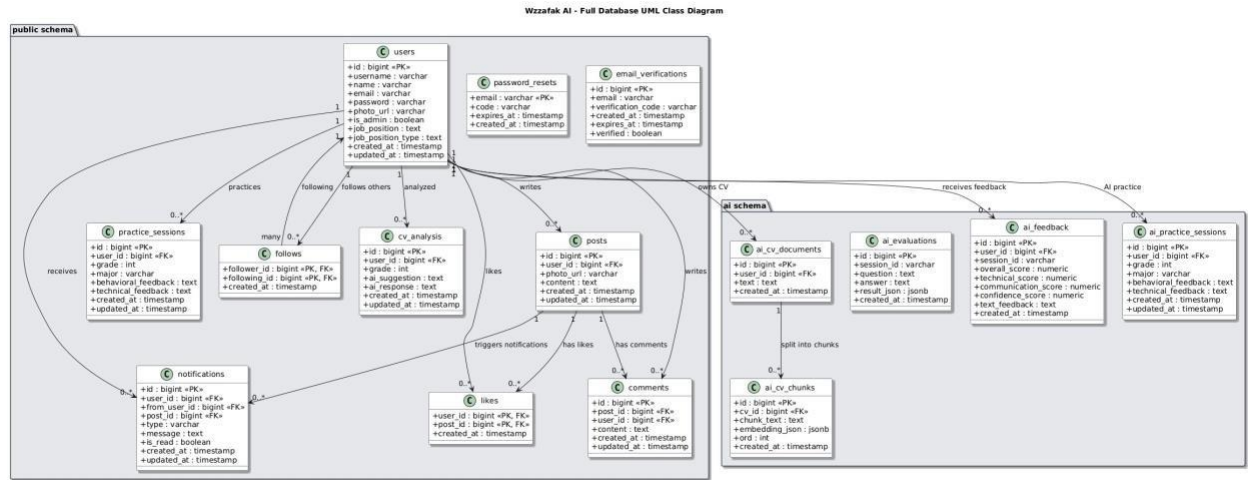


Figure 7: UML Class Diagram

The class diagram presented in Figure illustrated the complete structural design of the wazzafak AI system , which works on 2 schemas , the public schema that handles user and social aspects of the platform , and the ai schema that handles intelligent interview analysis and feedback components.

The public Schema

The **user class** centers this schema, maintaining core profile, and authentication information, it also connects to several other entities representing the social and engagement features of this application such as:

- **Posts, Comments, Likes and Follows:** Within these classes, users can interact with other users by posting content (photos with captions), and engage with them even more deeply by following them.

- **Notifications:** Generated when users follow other users or interact with their posts to enable real time updates.
- **Practice Sessions, and CV Analysis:** These tables store user cv analysis and practice interview results, which act as a resource for feedback and performance tracking.

These relationships reflect the interactive nature of the platform and enable community learning through shared feedback.

The ai Schema

- The AI schema defines all system components involved in conducting automated interview assessments and performing CV analysis.
- **CV Documents and CV chunks:** CVs are stored as full documents and broken into smaller chunks with embeddings, allowing AI models to perform semantic understanding and matching
- **AI Evaluations:** Every AI generated question is recorded along with it's answer . Even if they are not shown in the front end, or user dashboard, they are used to ensure analysis and recommendations are well studied .
- **AI Feedback:** All feedback provided, including answer feedback or sentiment scores are stored in this table.

These classes collectively support intelligent evaluation pipelines, enabling personalized and measurable improvement for users.

Chapter 4: Machine Learning Design

Although the system incorporates multiple AI models, machine learning plays a significant role in enabling intelligent analysis and feedback.

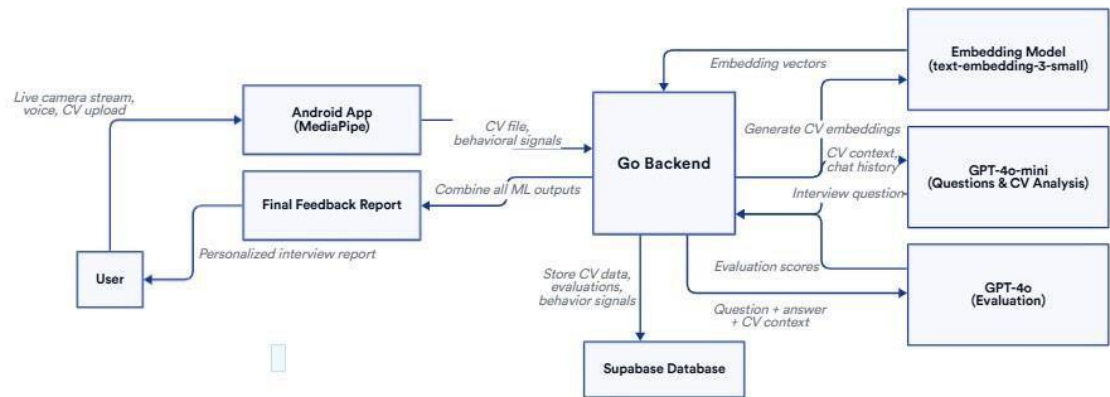


Figure 8: Machine Learning Workflow

The process begins when the user uploads their CV or submits responses during the interview. The Android app acts as the entry point, sending the CV file, the user's voice responses, and the behavioral signals captured through MediaPipe directly to the Go backend. Once a CV is received, the backend extracts its text, segments it into chunks, and sends those chunks to the embedding model (text-embedding-3-small). The model returns embedding vectors that represent the CV in a semantic way, enabling the backend to understand the user's skills, experience, and domain more intelligently.

These embeddings, along with the CV context and conversation history, are then passed from the backend to GPT-4o-mini. This model is responsible for generating context aware interview questions and performing the initial CV analysis. During the interview, every answer the user provides is sent back to the backend, which forwards the question, the answer, and the relevant CV context to GPT-4o.

GPT-4o evaluates the response and returns detailed scoring and reasoning. Meanwhile, the behavioral signals gathered from MediaPipe on the Android device, such

as facial movement, posture deviation, and eye-contact consistency are recorded and stored in the Supabase database alongside CV data and evaluation results.

On the behavioral side, the Android device runs a lightweight ML pipeline using Google's pretrained MediaPipe models for both face tracking and pose detection. These models operate directly on the live camera feed and output a set of high resolution facial and skeletal landmarks. The face mesh model enables the app to compute the Eye Aspect Ratio (EAR) to determine whether the user's eyes are open, and it identifies gaze direction by tracking the displacement of eye center landmarks relative to the center of the screen. This is how the system detects loss of eye contact or downward glances. In parallel, the pose model analyzes shoulder alignment and hip positioning to estimate posture quality, classifying behaviors such as leaning too far forward or backward. Combining these measurements makes it possible for the system to detect nonverbal signals typically associated with anxiety or disengagement, including head tilting, slouching, and inconsistent eye contact.

As the interview progresses, these behavioral features are continuously computed on device and sent back to the backend as structured signals, complementing the linguistic evaluations produced by GPT-4o.

Although MediaPipe operates independently from the language models, the features it extracts which are EAR values, gaze deviation, shoulder imbalance, and lean angle, form a crucial layer of the final analysis by quantifying the user's physical presence and attention during the interview.

When the interview ends, the backend pulls together all ML outputs: the CV embeddings, the generated questions, the evaluation scores from GPT-4o, and the behavioral cues from the app. These elements are combined into the Final Feedback Report,

which is then returned to the user as a personalized summary of their technical performance, communication quality, and behavioral presence throughout the interview.

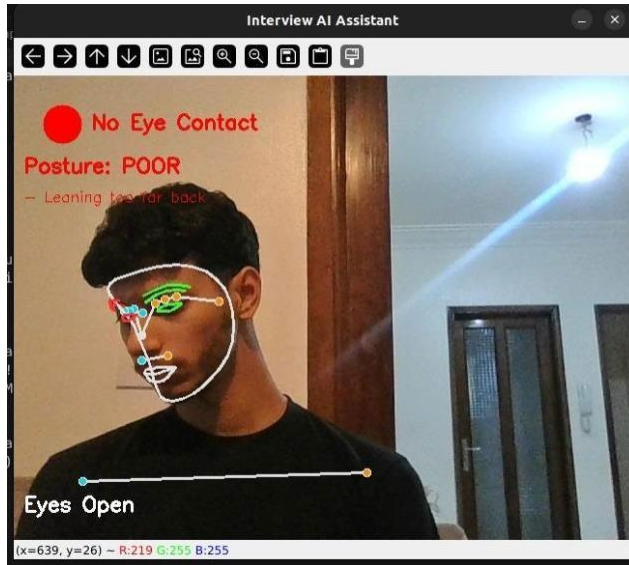


Figure 9: Sentiment analysis model

Chapter 5: System Implementation

This chapter elaborates on the choices of logic , flow and implementation of Wazzafak , it also offers a step by step insight of how the app started from scratch , then became two separate source codes , and finally , how they were integrated .

At first , Wazzafak was an Android App by itself with its own backend , and an Ai interview logic was being developed alone with it's own backend and docker container.

Once both parts were fully furbished , they entered a deep integration process which ended up putting them under one server and one database.

5.1 The Android App

The Android application was developed using Kotlin and Jetpack components.

The Android application serves as the primary user interface, enabling users to upload their CVs, perform live AI-simulated interviews, and view personalized technical,

behavioral, and CV quality feedback. The mobile application adopts a Model–View–ViewModel (MVVM) architecture to ensure separation of concerns, lifecycle stability, testability, and maintainability.

A high-level overview of the mobile architecture is presented in Figure

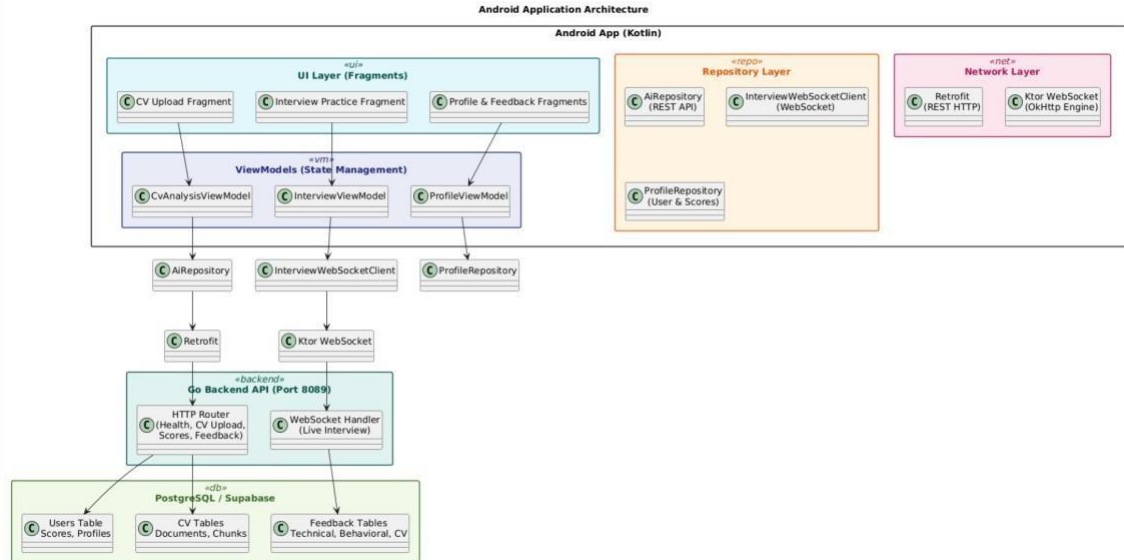


Figure: 10 Android Application Architecture

5.1.1 Authentication, Tokens, and Email Verification

One of our first, and most important implementations, which ties the whole experience together. It is the authentication and account security flow.

In our application, we implemented a full email-based verification flow to ensure that every user account is genuine and securely tied to a valid email address. When a new user signs up or requests a password reset, we do not immediately treat the account as fully trusted. Instead, our backend generates a secure, time-limited verification token, stores it in the database, and sends an email containing a unique verification link to the user. Only after the user clicks this link, and the backend successfully validates the token, do we mark the account as verified and allow normal access to the system. On the Android side, the process feels simple: the user fills in the registration form, submits it, and then is prompted

to check their email. However, behind the scenes, we are enforcing strict checks so that interview results, CV analyses, and social features are only associated with confirmed, real users rather than temporary or fake accounts. This verification layer is especially important for our AI features, because it protects us from noisy, anonymous traffic and ensures that all the data we use for scoring and feedback actually belongs to authenticated, verified people.

we generate a secure, time-limited verification token using a cryptographically strong random generator and store it in a dedicated table along with the user's ID and an expiration timestamp. This token essentially acts as a temporary key that proves the request is real. Once generated, we embed it into a verification link such as <https://wazzafak.app/api/v1/auth/verify?token=<token>> and send it to the user through our email service.

The two figures below , respectively represent the users receive upon the creation of a new account , and the email users receive upon trying to reset their password.

From the user's perspective, all of this is invisible; the only thing they see is a “please check your inbox” message. But once they click the link, the backend finally performs the important part. The `/v1/auth/verify` endpoint reads the token from the URL, looks it up in the database, and validates that it is still active, not expired, and belongs to the correct user. If everything checks out, we flip the user's `is_verified` flag to true and mark the token as used so it cannot be exploited again. The logic is compact but strict:

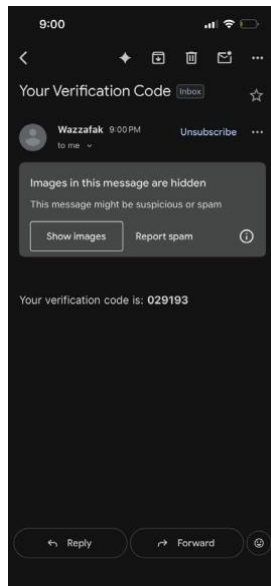


Figure 11: Sign up verification email

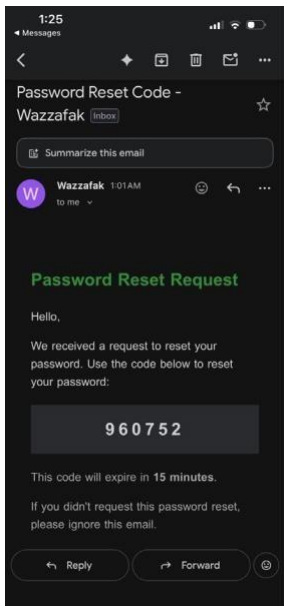


Figure : Reset Password Verification email

After the user is verified , they receive a valid JWT, which the Android app will attach to every protected request, including CV uploads, interview sessions, score retrieval, and feedback fetching. This verification step prevents fake sign-ups, protects the system from anonymous misuse, and guarantees that every CV, every interview, and every evaluation stored in our database belongs to an actual confirmed user.

By enforcing email verification at the backend and handling it properly through token generation, storage, validation, and cleanup, we make sure the entire AI pipeline operates on clean, trustworthy user identities. This creates a secure foundation for everything else in the app, and it is one of the reasons the rest of the system stays stable.

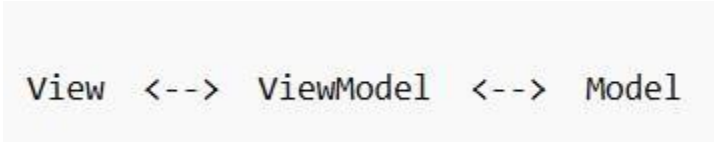
5.1.2 UI Architecture and MVVM Structure

The android application is divided into independent feature modules aligned with MVVM principles.

An MVVM structure is a common architectural pattern which have been recently used in many types of applications , but mostly Android , Flutter , .NET MAUI , etc

It basically seperates the code into 3 structures :

1. Model which contains the API responses , DB , and is the business logic and no UI code
2. Only UI and no logic , all it does is display data from ViewModel
3. ViewModel,the middle layer and communicator between the 2 above , so the middle layer between the view , and the model. It communicated with the model and prepares data for the view. It also deal with UI logic and state management.



```
View <--> ViewModel <--> Model
```

Figure 13: Communication of MVVM

MVVM also enables a reactive user interface, where Fragments simply observe the ViewModel; whenever the underlying data changes—such as updated scores or new interview messages—the UI updates automatically without requiring

manual checks. In addition, the architecture enforces clean separation of concerns. Fragments handle only the visual presentation, while all business logic, network operations, and data processing are delegated to ViewModels and repositories. This prevents UI classes from becoming overloaded and makes the codebase significantly easier to maintain. Finally, placing logic inside ViewModels rather than Fragments makes the system highly testable, since the core functionalities can be validated through unit tests without relying on Android-specific components. Together, these characteristics make MVVM an ideal architecture for a complex, interactive application such as Wazzafak AI.

UI components:

- ***CVFragment*** : Allows uploading and analyzing the user's cv
- ***PracticeFragment***: used for conducting real time interviews using Speech recognition, TTS, and camera analysis
- ***FeedbackDetailFragments***: Displays detailed feedback categorized by domain

5.1.3 Networking Layer: Retrofit and Ktor WebSocket Two

separate communication mechanisms were implemented:

1. **Retrofit (REST API) configured with:**

- 15 minute timeouts to accommodate long CV processing
- Logging interceptors for debugging network failures
- Automatic retries for intermittent connectivity

Retrofit is responsible for handling end point , the figure below indicates some of the endpoints it handles .

Endpoint	Purpose
POST /v1/cv/upload	Upload CV for extraction and analysis
GET /v1/user/scores	Retrieve updated user scores
GET /v1/user/feedback/*	Fetch detailed CV, technical, and behavioral feedback
GET /v1/health	Connectivity check

Figure 14: REST API endpoints

2. Ktor WebSocket Client (Real time interview):

Live interview sessions rely on a persistent WebSocket connection established via Ktor's OkHttp engine.

We implemented the following key features to elevate the app's quality:

- 20 second automatic ping interval to maintain socket health.
- SharedFlow for emitting interview messaged to the UI.
- Automatic reconnection logic on unexpected disconnection.
- Full duplex communication , meaning the backend continuously

sends AI generated questions to the application while the android client simultaneously returns the user's audio- to-text answers in real time . This exchange allows a low-latency, synchronous interview interactions that resemble a natural conversational flow.

5.1.3 CameraX and ML Kit Integration :

This is how we implemented real-time behavioral assesement , also known as the practice module.

Using the CameraX API , each frame in the front facing camera is captured and processed through ML kit's face-detection and pose-detection models, which lets the system interpret the user's eye contact, shoulder alignment , and overall posture. These frames are analyzed roughly every 500 milliseconds, which provides continuous behavioral insight without imposing excessive computational or battery overhead on the device.

For example, during an interview, when the AI interviewer sends a new question over the WebSocket connection, and the Text-to-Speech engine finishes speaking, the user will be answering aloud while the camera simultaneously tracks whether they maintain eye contact or appear distracted. As the *SpeechRecognizer* transcribes the response, ML Kit may detect that the user repeatedly looks away during difficult questions or shifts their posture significantly when uncertain. These signals reflect natural behavioral tendencies observed in the workflow, such as moments when the user pauses before answering, changes head orientation while thinking, or leans back when unsure.

They can later contribute to the broader behavioral feedback generated after the interview session. In this way, the ML-based analysis is fully synchronized with the interview flow, enriching the user's performance evaluation with subtle but meaningful non-verbal behavioral data.

5.1.4 Speech Recognition and Text-to-Speech Integration:

The speech system in our application was designed not only to behave as close as possible to a natural interview, but also to avoid the technical pitfalls that we encountered in early prototypes. At first, the *SpeechRecognizer* would start listening immediately when the screen loaded, even while the AI was still speaking through Text-to-Speech, which caused the system to mistakenly capture the AI's own voice as user input. In some cases, the app began answering its own questions. We also noticed that background noises, such as rustling, fans, or distant voices, were sometimes interpreted as speech, which led to incorrect or incomplete answers being sent to the backend. To solve these issues, we implemented tight control over when the recognizer is allowed to listen by linking it directly to the *TextToSpeech* lifecycle.

Using an *UtteranceProgressListener*, the app disables speech recognition the moment TTS begins speaking, and only re-enables listening once the AI finishes its question.

In technical terms, this means recognition only starts inside `onDone(utteranceId)`, guaranteeing that no overlap occurs.

We also added silence detection, where the recognizer waits for a short quiet period before finalizing an answer, preventing premature cutoffs. Additionally, automatic retry logic ensures that if noise or unclear audio triggers an error, the recognizer restarts smoothly without breaking the interview flow.

Together, these design choices allow us to maintain a clean separation between AI output and user input, protect the system from noisy data, and provide a stable, realistic conversational rhythm where the AI speaks clearly, and the user responds without interference.

5.1.5 Interview Session Control Flow

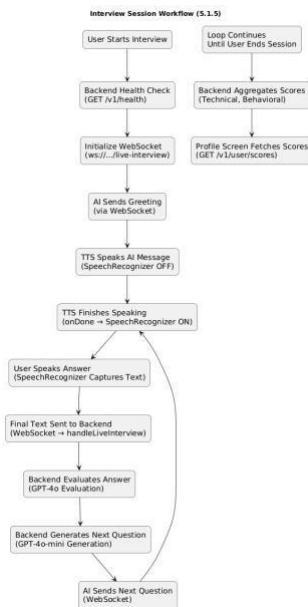


Figure 15: Interview flow

In our implementation, which is shown in Figure 15, the interview session runs in a smooth loop where each step naturally triggers the next. We start by checking that the backend is reachable, then we open a WebSocket connection which allows the AI to send its first greeting. As soon as the AI speaks through TTS, we disable the SpeechRecognizer to prevent it from capturing the AI's own voice. Once TTS finishes, we turn listening back on and the user can answer normally. Their final transcribed text is sent to the backend, where the answer is evaluated

and the next question is generated. The cycle repeats: the AI speaks, TTS finishes, we listen, we send the response, and the backend returns another question. This loop continues until the user ends the session, after which the backend aggregates the scores and the profile screen retrieves the updated results. The entire flow is designed to feel natural while keeping strict control over when the system listens and speaks.

5.1.6 CV Upload :

In our Android app, the CV upload flow is handled inside `cvfragment.kt`, and everything is wired through our `CvAnalysisViewModel` to keep the UI clean and responsive. When the user taps the upload button, Android opens the system file picker and restricts the selection to PDF and DOCX files, matching the supported formats on the backend. Once the user chooses a document, the fragment passes the URI to the ViewModel, which resolves it using `contentResolver.openInputStream(uri)` because Android does not provide direct file paths for external storage. The ViewModel then reads the file into a byte array and wraps it in a `MultipartBody.Part` so Retrofit can send it to the backend. Since CVs can be large, we configured our OkHttp client with 15-minute read and write timeouts, exactly as shown in `AiRetrofitClient.kt`, to ensure that slow networks or heavy CV parsing on the backend do not break the upload. The ViewModel triggers the upload inside a coroutine, keeping the UI thread free, and the fragment shows a loading indicator until the request completes.

After the POST request reaches `/v1/cv/upload`, we simply wait for the backend to finish processing. The fragment then receives the response through the ViewModel, displays the returned score and summary, and finally navigates back to the Profile screen, where the updated CV score is fetched automatically.

The analysis process is described later in section 5.2, but this part of the system ensures that the uploaded file reaches the backend reliably and consistently across all Android devices.

5.1.7 User Profile and Score Retrieval:

The Profile screen is the user dashboard where users can see how they are doing across the different aspects of the system. Each time the user navigates to the Profile screen, the app automatically triggers a call to the backend to fetch the latest values for the three main scores: the Technical Score, the Behavioral Score, and the CV Analysis Score. These values are stored in the *ai.feedback* table on the backend and are updated after each interview session and after each CV upload.

On Android, the *ProfileViewModel* issues a GET `/v1/user/scores` request (using the currently authenticated user ID), and once the JSON response is received, it updates the state that the Fragment is observing. The UI then renders each score inside its own card, making it easy for the user to see at a glance how their technical answers, behavioral performance, and CV quality are evolving over time. If the user has just completed an interview or uploaded a new CV, returning to the Profile screen immediately reflects those changes, which closes the loop between interaction, AI processing, and user feedback in a way that feels responsive and meaningful rather than static.

5.1 Reflection

The implementation of the Android application highlighted the importance of structuring the client-side logic in a way that keeps the user experience seamless while managing complex background operations. Each module, ranging from authentication and CV uploads to live interview interaction, demonstrated how essential it is to balance UI responsiveness with strict technical requirements such as long running network calls, token validation, and controlled audio processing.

By relying on ViewModels, coroutines, and clearly separated layers, we were able to maintain stability even when handling camera streams, speech recognition, or large file transfers.

5.2 The AI logic code

In our system, all AI-driven functionality, CV analysis, interview question generation, and answer evaluation run entirely on the Go backend, which acts as the intelligence layer of the application. The Android app simply sends data to the server, and all reasoning, scoring, and interpretation happen on the backend through a tightly defined pipeline. This separation allows us to run heavyweight operations such as PDF parsing, text chunking, embedding generation, and GPT-based evaluation without placing any burden on the mobile device.

The graph below, labeled Figure , shows a simplified but detailed step by step process of the CV chunking and analysis pipeline.

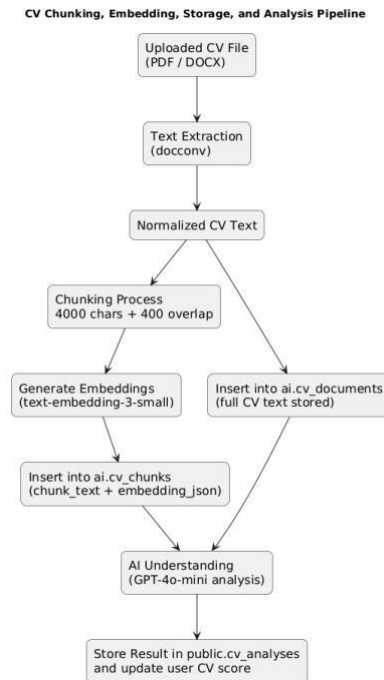


Figure16 :CV analysis pipeline

5.2.1 CV Analysis Pipeline (Backend AI Stage)

Once the Android client successfully uploads the user's CV, the backend begins a multi-stage analysis pipeline implemented in `cv_parser.go`. The raw

document is first processed through docconv, which extracts text from PDF, DOCX, and other supported formats. Because CVs rarely follow a clean structure, we normalize the extracted text and split it into overlapping 4,000-character chunks with a 400-character overlap. This ensures that the later embedding process preserves context across section boundaries. Each chunk is then passed to the OpenAI text-embedding-3-small model, and the resulting vectors are stored in the ai.cv_chunks table alongside the original CV text stored in ai.cv_documents. The backend also performs a first-pass AI understanding using GPT-4o-mini, which identifies the candidate's field, strengths, weaknesses, and missing elements in the CV. That result is saved in public.cv_analyses and the users.cv_analysis_score is updated accordingly. All processing is done synchronously inside the request lifecycle, which is why the Android client uses long timeouts.

5.2.2 Live Interview Intelligence Engine

The real complexity of our AI system appears during live interviews, where communication between the Android client and backend happens over WebSocket through the /v1/live-interview endpoint.

As soon as the session starts, the backend loads the user's CV embeddings and constructs a system prompt that blends CV-based context with the interview domain. This ensures that 50–70% of the questions reference the actual content of the user's CV.

Every AI question is generated in real time using GPT-4o-mini, chosen for its speed and cost efficiency.

When the user sends an answer back through the WebSocket, the backend passes the question–answer pair into the evaluation pipeline implemented in evaluateAnswerInternal().

If mock mode is off, the backend queries the knowledge base embeddings (ai.kb_articles) to extract relevant context for the field the user belongs to. This

context, along with the question and the user's answer, is fed into GPT-4o, which returns a JSON structure containing: an overall score, technical score, communication score, confidence score, a strengths list, and a weaknesses list.

Each result is stored in the `ai.evaluations` table with the session ID, allowing the backend to maintain a chronological record of the user's decisions and performance throughout the interview.

5.2.3 Feedback Aggregation and Score Computation

When the interview ends, the backend brings together every answer the user gave throughout the session and processes them as one complete performance. Instead of treating each evaluation separately, the system consolidates all the technical, communication, and confidence scores generated during the interview and calculates the final three metrics that appear in the user's profile. At the same time, the backend studies all the JSON outputs generated by GPT-4o for every question and identifies the patterns that define the user's behavior like recurring strengths, repeated weaknesses, and tendencies that appeared more than once. These elements are then woven into one unified feedback message that feels coherent, detailed, and genuinely connected to the way the user responded. Once this combined result is ready, it is saved in the `ai.feedback` table, and the relevant final scores are updated in the `users` table so the Android profile screen can immediately display them.

5.2 Reflection

This backend driven AI architecture allows us to keep all heavy computation away from the mobile device. The phone does not parse files, does not evaluate answers, and does not store embeddings. The Android app simply streams data back and forth, and the backend handles intelligence, structure, and scoring. This gives us the freedom to enhance AI quality over time without requiring users to update

their app. It also ensures that every interview, no matter how long it is , stays synchronized, consistent, and securely tied to the correct user session.

5.3 Integration

The integration between the Android client and the Go server forms the backbone of the entire system. Although the user interacts only with a simple interface, nearly every action inside the app results in structured communication with the backend. This connection is built on two channels: traditional REST endpoints for file uploads and score retrieval, and a persistent WebSocket channel for real-time interview sessions. Together, these channels allow the system to deliver fast AI feedback, stream live interactions, manage authentication, and maintain accurate user records without exposing any complexity to the user.

5.3.1 REST Integration

All standard operations such as uploading a CV, fetching scores, and retrieving feedback are handled through REST using Retrofit. On the Android side, each request is defined in `AiService.kt` and executed from a `ViewModel` to keep UI logic clean. We configured `OkHttp` with extended 15-minute timeouts to support large CV files and slow networks. Incoming requests reach the Go server's HTTP router, where endpoints like `/v1/cv/upload` and `/v1/user/scores` perform the required operations before returning structured JSON responses.

This REST channel is responsible for all non-live features and ensures stable, predictable communication between the two sides

5.3.2 WebSocket Integration for Live Interviews

The most dynamic part of the system is the live interview session, where communication between the app and backend must be continuous. The Android app opens a WebSocket connection using Ktor, while the server upgrades the connection using Gorilla WebSocket. Once connected, the backend loads the user's CV context, constructs a system prompt, and sends the first AI message.

- The interview flow becomes a tight loop:
- Backend sends a question
- TTS speaks it
- TTS finishes → SpeechRecognizer enables
- User answer captured and streamed back
- Backend evaluates the answer using GPT-4o
- Backend generates the next question using GPT-4o-mini

Loop repeats

5.3.3 Authentication and Token Handling

Authentication is integrated into both REST and WebSocket flows. After login, the backend issues a JWT token. The Android app stores this token and automatically includes it in every protected request through an OkHttp interceptor.

On the backend, each endpoint extracts the token, validates it, and resolves the authenticated user. This ensures that all CV uploads, interview sessions, and stored results are linked to the correct user and cannot be forged.

5.3.4 Data Round-Trip and State Synchronization

Even though the processing occurs on the backend, the Android app must always display up-to-date information. After a CV upload or interview session ends, the app automatically calls `/v1/user/scores` to refresh the profile screen. This ensures immediate synchronization between the AI outputs and the user interface, giving the impression of a live, responsive system.

Example:

CV uploaded → backend processes → ViewModel updates →
ProfileFragment shows new score

Interview ends → backend compiles feedback → Android fetches scores →
UI refreshes automatically

This constant exchange keeps the system consistent and prevents stale or mismatched data from appearing on the client.

5.3 Reflection

The integration work showed that the Android app and the Go backend depend on each other in very clear and practical ways. The REST API handles tasks like uploading CVs and retrieving scores, while the WebSocket connection supports the live interview by allowing questions and answers to move instantly between both sides. Token-based authentication keeps everything secure without adding extra steps for the user. After each major action, such as finishing an interview or uploading a CV, the app immediately requests the updated scores to stay consistent with the backend. Overall, the integration behaves exactly as intended: the Android

app sends structured data, the backend processes it, and the results return smoothly to the UI.

It is a reliable system where both components work together to keep the experience stable and accurate.

5.4 Routing And Endpoints

The following table represents the most important routing endpoints used in our application:

5.5 Application pages

In this section , we visually explore the flow of our application, and the UI over all what we have previously discussed in chapter 5.

Enpoint	Method	Purpose
/v1/health	GET	Backend status check
/v1/cv/upload	Post	Upload CV (multipart)
/v1/live-interview	WebSocket	Real-time interview session
/v1/user/scores	GET	Retrieve user scores
/v1/user/feedback/technical	GET	Technical interview feedback
/v1/user/feedback/behavioral	GET	Behavioral interview feedback
/v1/user/feedback/*	GET	Full session details

/v1/auth/register	POST	Register user + send verification email
v1/auth/login	POST	Login + token creation
/v1/auth/reset-request	POST	Send reset password email
/v1/auth/reset	POST	Complete reset
/v1/auth/verify	GET	Email verification

5.5 Application pages

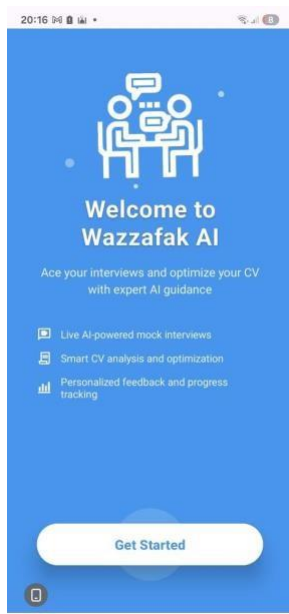


Figure 17: Splash/Welcome Screen

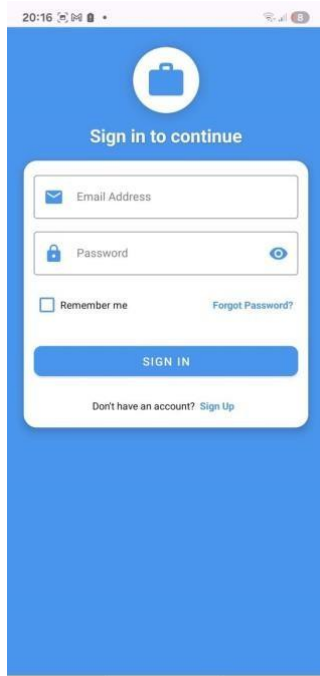


Figure 18: Login page

Login screen is the UI interface covering the token authorization we previously discussed, and it includes the options to hide or show password , save password for later on , change forgotten passwords, ,or go to sign up page.

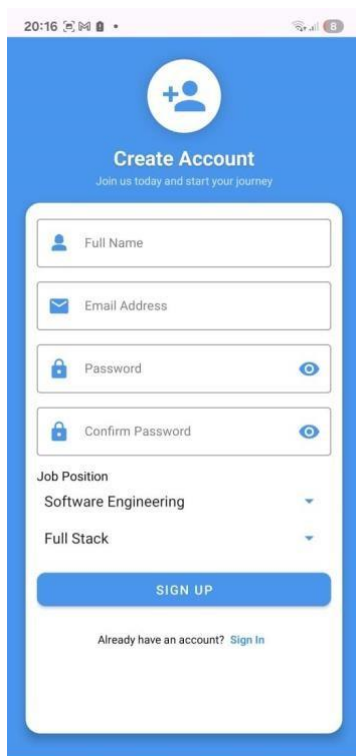
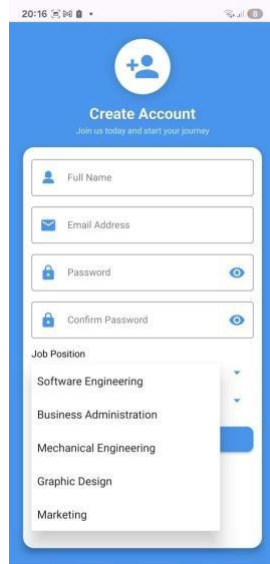


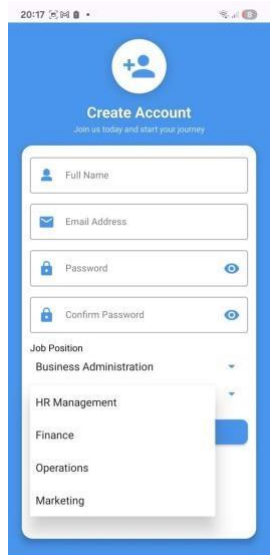
Figure: Sign Up page

The sign up page is the page where email authorization happens, it also offers user the chance to choose their field in case it was one of the supported field and the specific domain in that field.



A screenshot of a mobile application's 'Create Account' screen. The screen has a blue header with a white user icon and the text 'Create Account' and 'Join us today and start your journey'. Below the header are four input fields: 'Full Name', 'Email Address', 'Password', and 'Confirm Password'. Each field has a corresponding icon (person, envelope, lock, and lock with eye) and a toggle eye icon. Below these fields is a 'Job Position' dropdown menu. The dropdown is open, showing a list of options: 'Software Engineering', 'Business Administration', 'Mechanical Engineering', 'Graphic Design', and 'Marketing'. The 'Marketing' option is highlighted with a blue bar.

Figure 20: Field choices



A screenshot of the same 'Create Account' screen, but with the 'Job Position' dropdown menu set to 'Business Administration'. A new dropdown menu is open, showing a list of domain choices: 'HR Management', 'Finance', 'Operations', and 'Marketing'. The 'Marketing' option is highlighted with a blue bar.

Figure 21: Domain choices Example

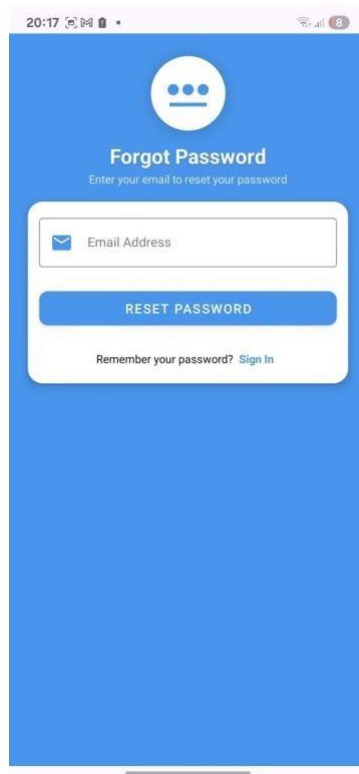


Figure 22: Forgot password

When reset password button in this page is pressed it triggers the function responsible for sending email verification codes for resetting passwords , email is shown in figure , once approved a pop up comes up for the user to set a new password.

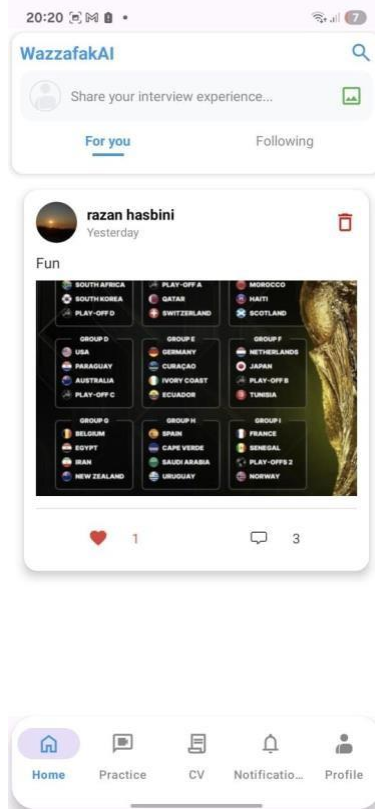


Figure 23: Home page

This page is responsible for showing the user's feed. The feed is split into 2 sections , following and For you.

The For You Feed is like an explore page for users to discover people they don't know's experiences, while the following page appends the posts of only the people whom they follow. Even though this feed is used for sharing interview practice experiences, the application logic does not set any restrictions on what users post .

The photo shown in the feed above is an example. Users can also like and comment on the posts shown in the feed , post their own by pressing the green image icon or share your experience bar , and even search for other users to follow by pressing on the blue search icon at the top.

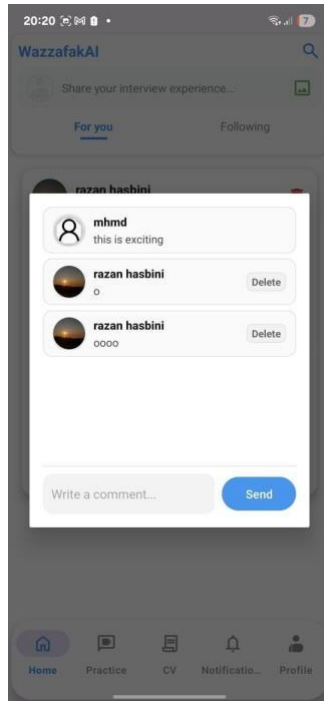


Figure:24 Comments on posts

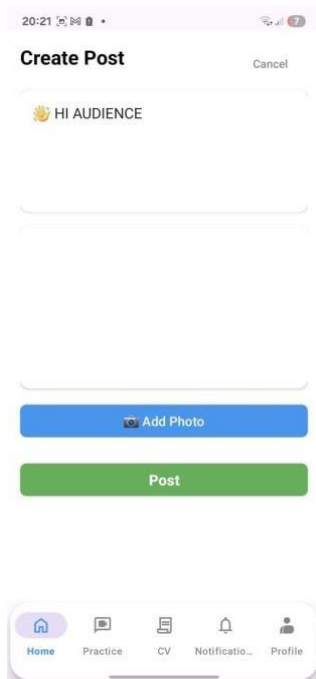


Figure 25: Creating a post page.

On this page , users can access their gallery by pressing on Add Photo , to choose an image , post without an image.

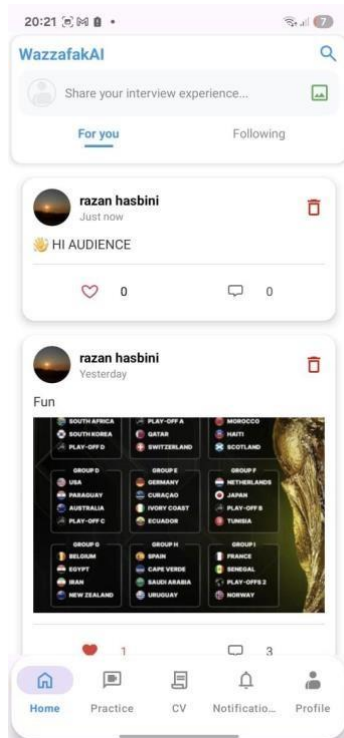


Figure 26 :Post success

Upon posting , the feed directly and automatically updates with the user's post in it , for the user himself , and other users. Then the user also has the choice to delete the photo by pressing on the delete button, and it directly is removed off the feed.

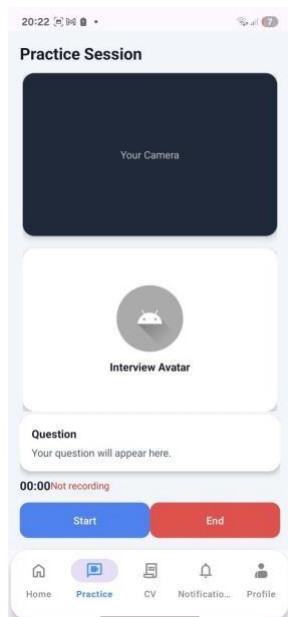


Figure 27: Live Interview page

At the beginning of this page, the camera and mic are blocked due to security settings, when start is pressed, the user should approve on the app connecting to both, then the interview starts.

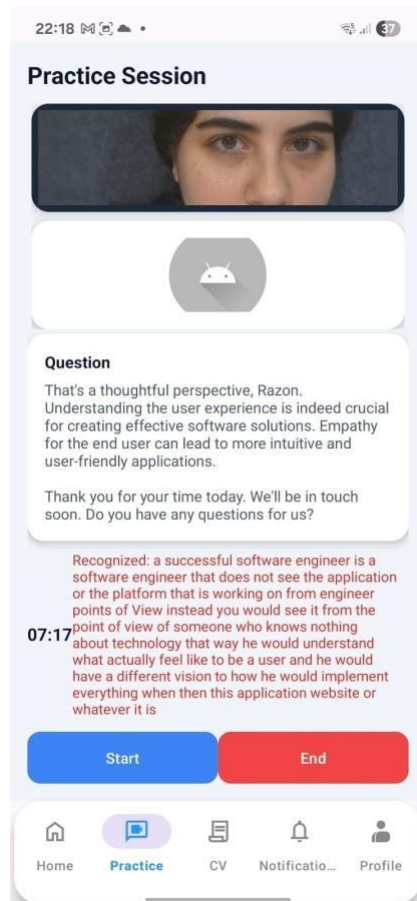


Figure 28: Interview snippet



Figure 29: Notifications Page

Notifications are sent to the user on this page whenever another user follows them, or interacts with their posts.



Figure 30: CV Feedback on CV screen

In this page , users can go into their saved documents to select a PDF or DOCX as a CV , and analyze.

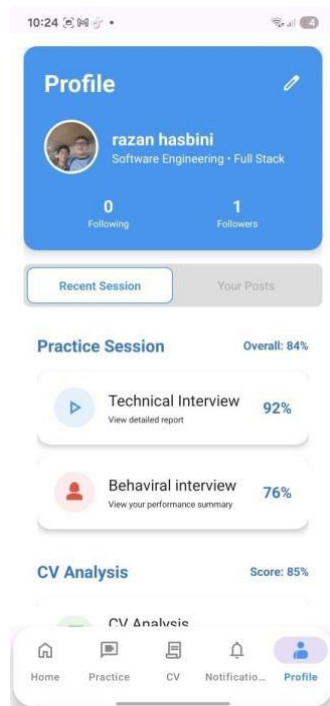


Figure 31: Profile Screen

This is the page that allows users to edit their credentials ,profile picture, and view , cv analysis skills percentage result , plus the detailed results of their interview by pressing on the fragment related to it.



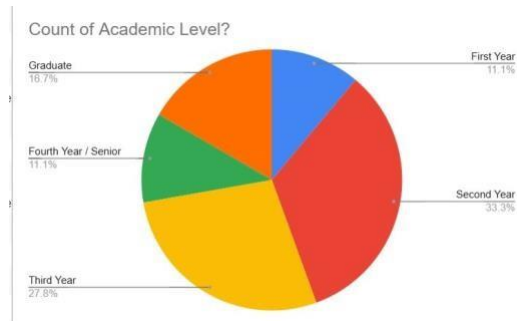
Figure 31: Bottom Navigation bar Fragment

Allows users to smoothly navigate between pages upon their presence in any of the pages.

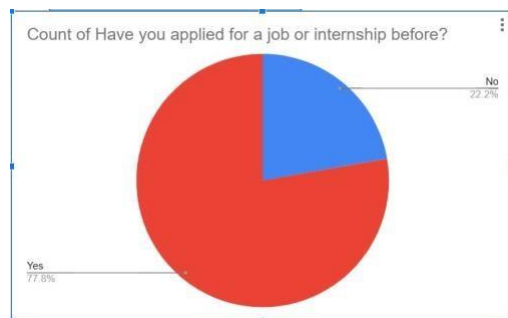
Chapter 6: Survey and Evaluation

In order to ensure an app like Wazzafak would have proper engagement , and implement all the features we know people would want, we created a survey with the following questions and their results:

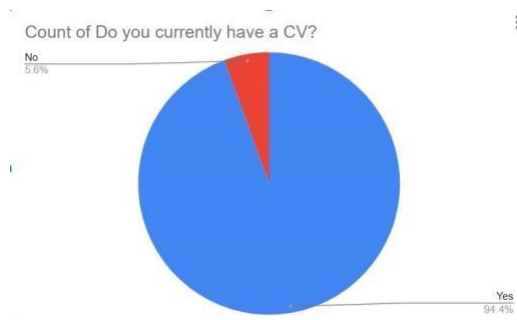
1) What is your academic Level?



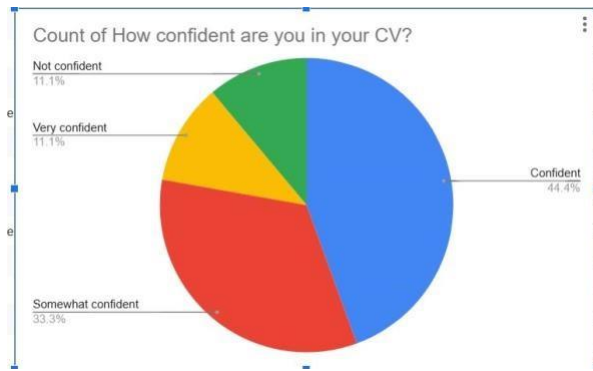
2) Have you applied for a job or internship before?



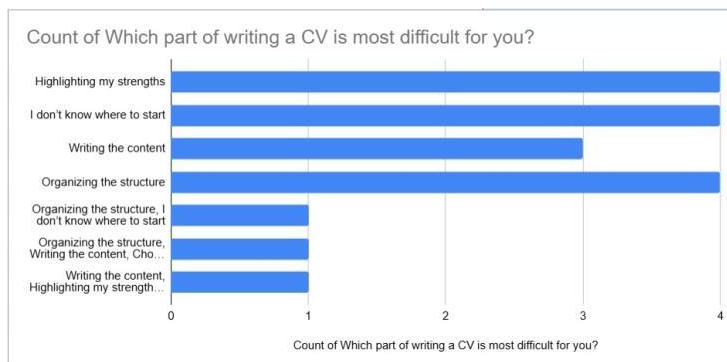
3) Do you currently have a cv?



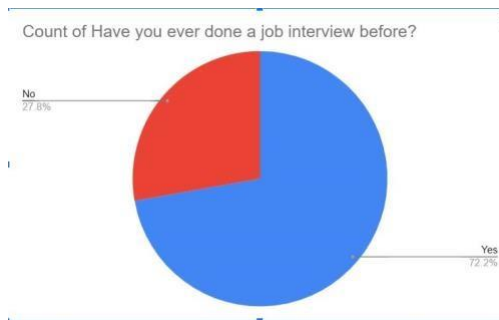
4) How confident are you in your cv?



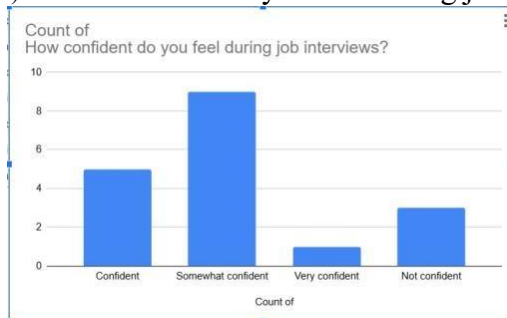
5) Which part of writing your cv is the most difficult for you?



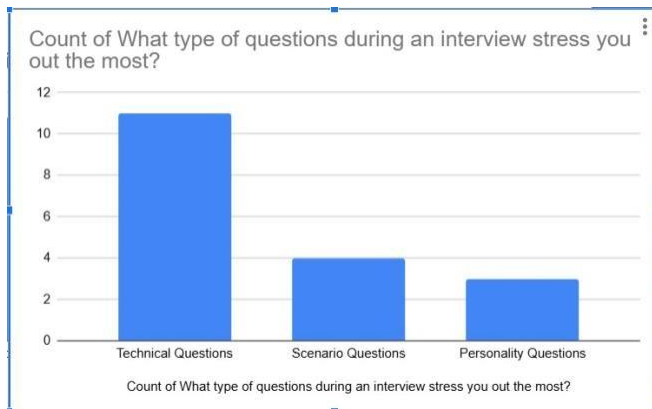
6) Have you ever done a job interview before?



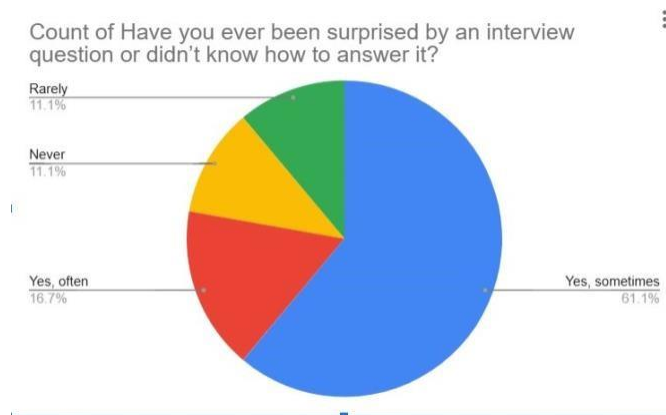
7) How confident do you feel during job interviews?



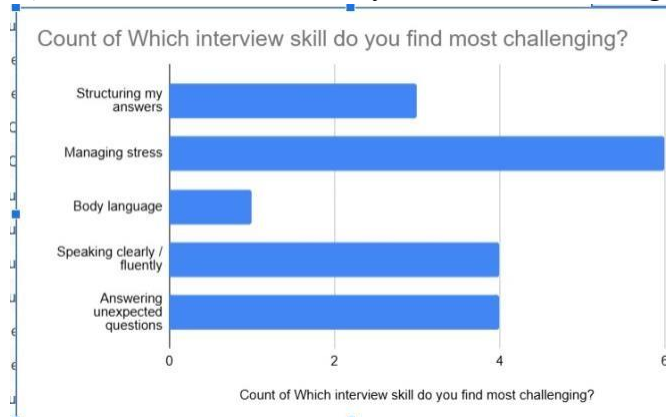
8) What type of questions during an interview stress you out the most?



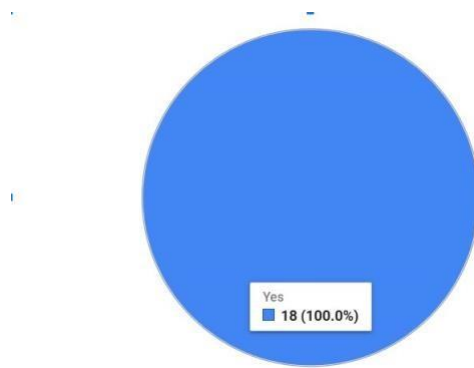
9) Have you ever been surprised by an interview question and did not know how to answer it?



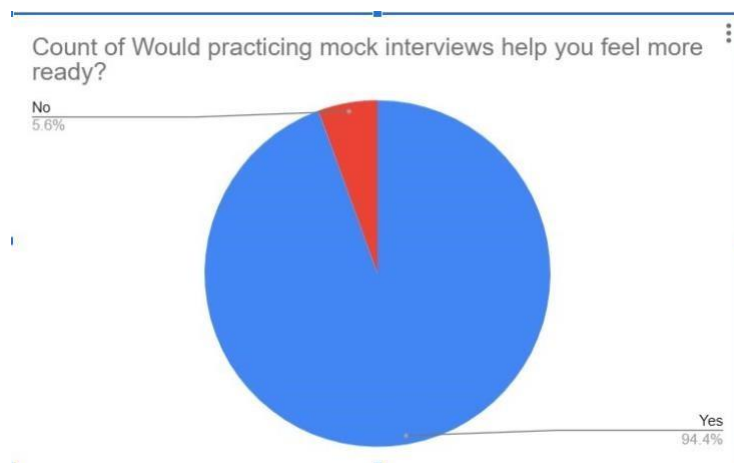
10) Which interview skill do you find most challenging?



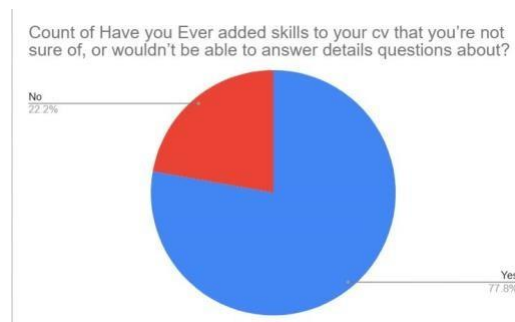
11) Would personalized feedback on your cv help you improve it?



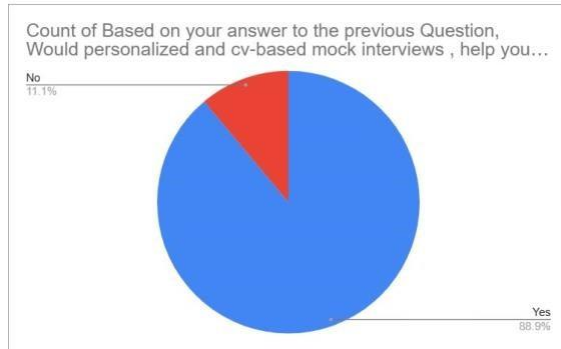
12) Would practicing mock interviews, help you feel more ready?



13) Have you ever added skills to your cv that you are unsure of, or would not be able to answer questions about?

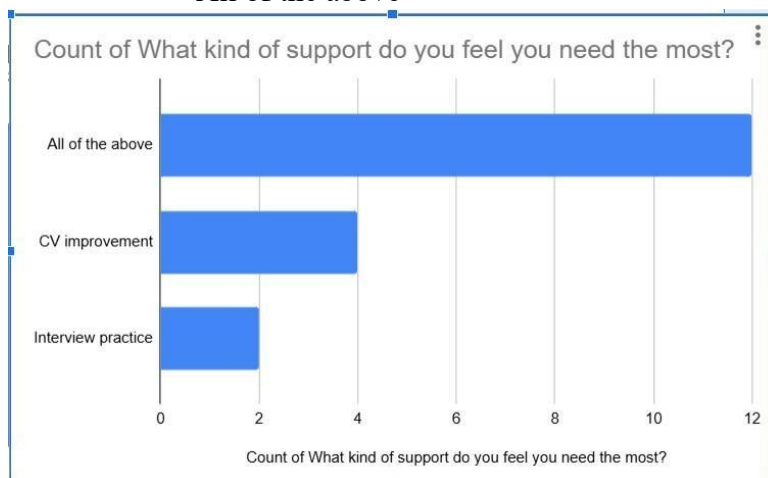


14) Based on your answer to the previous question, would personalized and cvbased mock interviews , help you overcome that fear?



15) What kind of support do you feel like you need the most? Multiple choice included:

- CV improvement
- Interview Practice
- Confidence Building
- All of the above



16) **Open Question nb 1**

Describe a situation where you felt unprepared during an interview , and what do you think would have helped you?

Summary of answers:

In general , most answers included situations of candidates going silent whenever they encountered a question they did not expect, and felt like they needed more confidence and skills that would have helped them recollect their thoughts on the spot.

17) **Open Question nb 2**

If an app could act as your personal job coach, what would you want it to do or say to help you feel truly prepared?"

Summary of answers:

Most popular reply was Constructive criticism on their replies, and emphasis on how they could have replied better or acted more professional.

Overall analysis:

After reviewing the results carefully, it was very clear that students and fresh graduates were in desperate need of an app like wazzafak AI , which tackles and goes over all possible types of personality , scenario , behavioral, and even technical questions that they might encounter, which further proves that our project is ought to succeed if introduced and published to the general community.

Chapter 7: Conclusion and Future Work

This project presents the design and implementation of an Android and Go-based system that establishes the foundational infrastructure for an AI-powered job interview training and CV analysis platform. The current implementation includes secure authentication, social interaction features, AI-driven interview analysis, and automated CV evaluation, all built on a modular architecture that separates concerns across both the mobile and backend layers. By leveraging MVVM on Android and a layered design on the backend, the system ensures scalability, maintainability, and a clear structure for continued development.

With the core features and AI components successfully implemented, the platform now provides users with practical tools that enhance their interview performance, improve their CV quality, and support their overall job readiness. The combination of social engagement, personalized AI feedback, and a structured user experience positions the system as a valuable resource for students and fresh graduates preparing for the job market.

Looking ahead, future enhancements will focus on creating several personalities for the AI, and creating stronger and smarter machine learning models integrated with datasets to elevate the quality of the interview and sentiment analysis.

We also plan to expand the platform's functionality through the integration of job opportunity listings, allowing users to browse and access relevant employment options directly through the application, as well as the development of a real-time chat system to support communication, mentorship, and professional interaction among users. These extensions aim to transform the system into a comprehensive career development ecosystem that connects users with opportunities and fosters meaningful professional growth.

References

- [1] Android Developers, "Activities, fragments, and architecture components," *Android Developer Guides*. <https://developer.android.com/guide>

[2] JetBrains, “Kotlin Coroutines: Official documentation,” *Kotlin Language Documentation*. <https://kotlinlang.org/docs/coroutines-overview.html>

[3] Square, “Retrofit: A type-safe HTTP client for Android & Java,” *Retrofit Documentation*. <https://square.github.io/retrofit/>

[4] Square, “OkHttp: HTTP & HTTP/2 client for Android and Java applications,” *OkHttp Documentation*. <https://square.github.io/okhttp/>

[5] The Go Programming Language, “Go official documentation,” *Golang.org*, <https://golang.org/doc/>

[6] Chi Router, “Lightweight HTTP router for Go,” *go-chi/chi*, <https://github.com/go-chi/chi>

[7] JSON Web Token, “JWT Introduction and Documentation,” *jwt.io*, <https://jwt.io/>

[8] Supabase, “Storage: Uploading, managing and accessing files,” *Supabase Documentation*, <https://supabase.com/docs/guides/storage>

Appendix A: Database Tables

Our database has 2 schemas , the public schema and ai schema. And is hosted on supabase.

Public schema:


```

updated_at timestamp without time zone DEFAULT CURRENT_TIMESTAMP,

CONSTRAINT comments_pkey PRIMARY KEY (id),

CONSTRAINT comments_post_id_fkey FOREIGN KEY (post_id)
REFERENCES public.posts(id),

CONSTRAINT comments_user_id_fkey FOREIGN KEY (user_id)
REFERENCES public.users(id)

```

```
);
```

```

CREATE TABLE public.cv_analysis ( id bigint NOT NULL DEFAULT

nextval('cv_analyses_id_seq'::regclass), user_id bigint NOT NULL, grade

integer CHECK (grade >= 0 AND grade <= 100), created_at timestamp without

time zone DEFAULT CURRENT_TIMESTAMP, updated_at timestamp without

time zone DEFAULT CURRENT_TIMESTAMP,

ai_suggestion text,

ai_response text,

CONSTRAINT cv_analysis_pkey PRIMARY KEY (id),
CONSTRAINT cv_analyses_user_id_fkey FOREIGN KEY (user_id)
REFERENCES public.users(id)

```

```
);
```

```

CREATE TABLE public.email_verifications ( id bigint

GENERATED ALWAYS AS IDENTITY NOT NULL,

email character varying NOT NULL, verification_code

character varying NOT NULL, created_at timestamp

without time zone DEFAULT now(), expires_at timestamp

```



```
without time zone NOT NULL, verified boolean

DEFAULT false,

CONSTRAINT email_verifications_pkey PRIMARY KEY (id)

);

CREATE TABLE public.follows ( follower_id bigint NOT NULL, following_id
bigint NOT NULL, created_at timestamp without time zone DEFAULT
CURRENT_TIMESTAMP, CONSTRAINT follows_pkey PRIMARY KEY
(follower_id, following_id),

CONSTRAINT follows_follower_id_fkey FOREIGN KEY (follower_id)
REFERENCES public.users(id),

CONSTRAINT follows_following_id_fkey FOREIGN KEY (following_id)
REFERENCES public.users(id)

);

CREATE TABLE public.likes ( user_id bigint NOT NULL, post_id bigint NOT
NULL, created_at timestamp without time zone DEFAULT
CURRENT_TIMESTAMP,

CONSTRAINT likes_pkey PRIMARY KEY (user_id, post_id),

CONSTRAINT likes_user_id_fkey FOREIGN KEY (user_id) REFERENCES
public.users(id),

CONSTRAINT likes_post_id_fkey FOREIGN KEY (post_id) REFERENCES
public.posts(id)

);
```

```

CREATE TABLE public.notifications ( id bigint NOT NULL DEFAULT
nextval('notifications_id_seq'::regclass), user_id bigint NOT NULL,
from_user_id bigint, post_id bigint, is_read boolean DEFAULT false,
type character varying NOT NULL, message text,
created_at timestamp without time zone DEFAULT CURRENT_TIMESTAMP,
updated_at timestamp without time zone DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT notifications_pkey PRIMARY KEY (id),
CONSTRAINT notifications_user_id_fkey FOREIGN KEY (user_id)
REFERENCES public.users(id),
CONSTRAINT notifications_from_user_id_fkey FOREIGN KEY
(from_user_id) REFERENCES public.users(id),
CONSTRAINT notifications_post_id_fkey FOREIGN KEY (post_id)
REFERENCES public.posts(id)
);

CREATE TABLE public.password_resets ( email
character varying NOT NULL, code character
varying NOT NULL, expires_at timestamp without
time zone NOT NULL,
created_at timestamp without time zone NOT NULL DEFAULT
CURRENT_TIMESTAMP,
CONSTRAINT password_resets_pkey PRIMARY KEY (email)
);

```

```

CREATE TABLE public.posts ( id bigint NOT NULL DEFAULT
nextval('posts_id_seq'::regclass), user_id bigint NOT NULL, photo_url
character varying, content text NOT NULL, created_at timestamp without time
zone DEFAULT CURRENT_TIMESTAMP, updated_at timestamp without time
zone DEFAULT CURRENT_TIMESTAMP, CONSTRAINT posts_pkey
PRIMARY KEY (id),

CONSTRAINT posts_user_id_fkey FOREIGN KEY (user_id) REFERENCES
public.users(id)

);

CREATE TABLE public.practice_sessions ( id bigint NOT NULL DEFAULT
nextval('practice_sessions_id_seq'::regclass), user_id bigint NOT NULL, grade
integer CHECK (grade >= 0 AND grade <= 100), major character varying,
created_at timestamp without time zone DEFAULT CURRENT_TIMESTAMP,
updated_at timestamp without time zone DEFAULT CURRENT_TIMESTAMP,
behavioral_feedback text,
technical_feedback text,

CONSTRAINT practice_sessions_pkey PRIMARY KEY (id),

CONSTRAINT practice_sessions_user_id_fkey FOREIGN KEY (user_id)
REFERENCES public.users(id)

);

```

```

CREATE TABLE public.users ( id bigint NOT NULL DEFAULT
nextval('users_id_seq'::regclass), username character varying NOT
NULL UNIQUE, name character varying NOT NULL, email
character varying NOT NULL UNIQUE,
password character varying NOT NULL,

photo_url character varying NOT NULL DEFAULT
'https://upload.wikimedia.org/wikipedia/commons/9/99/Sample_User_Icon.png'::characte
r varying, is_admin boolean DEFAULT false, job_position text NOT NULL,
created_at timestamp without time zone DEFAULT CURRENT_TIMESTAMP,
updated_at timestamp without time zone DEFAULT CURRENT_TIMESTAMP,
job_position_type text NOT NULL,

cv_analysis_score numeric DEFAULT 0 CHECK (cv_analysis_score >=
0::numeric AND cv_analysis_score <= 100::numeric),

CONSTRAINT users_pkey PRIMARY KEY (id)

);

```

ai schema:

```

CREATE TABLE ai.cv_chunks ( id bigint NOT NULL DEFAULT
nextval('ai.cv_chunks_id_seq'::regclass), cv_id bigint NOT NULL,
chunk_text text NOT NULL,
embedding_json jsonb,
ord integer,
created_at timestamp without time zone DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT cv_chunks_pkey PRIMARY KEY (id),
CONSTRAINT cv_chunks_cv_id_fkey FOREIGN KEY (cv_id) REFERENCES
ai.cv_documents(id)
);

```

```

CREATE TABLE ai.cv_documents ( id bigint NOT NULL DEFAULT
nextval('ai.cv_documents_id_seq'::regclass), user_id bigint NOT NULL, text
text NOT NULL, created_at timestamp without time zone DEFAULT
CURRENT_TIMESTAMP,
CONSTRAINT cv_documents_pkey PRIMARY KEY (id),
CONSTRAINT cv_documents_user_id_fkey FOREIGN KEY (user_id)
REFERENCES public.users(id)
);

```

```

CREATE TABLE ai.evaluations ( id bigint NOT NULL DEFAULT
nextval('ai.evaluations_id_seq'::regclass), session_id character varying
NOT NULL, question text NOT NULL, answer text NOT NULL,
result_json jsonb,

```

```

        created_at timestamp without time zone DEFAULT CURRENT_TIMESTAMP,

        CONSTRAINT evaluations_pkey PRIMARY KEY (id)

    );

CREATE TABLE ai.feedback ( id bigint NOT NULL DEFAULT

nextval('ai.feedback_id_seq'::regclass), user_id bigint, session_id

character varying NOT NULL, overall_score numeric, technical_score

numeric, communication_score numeric, confidence_score numeric,

text_feedback text,

        created_at timestamp without time zone DEFAULT CURRENT_TIMESTAMP,

        CONSTRAINT feedback_pkey PRIMARY KEY (id),

        CONSTRAINT feedback_user_id_fkey FOREIGN KEY (user_id)

REFERENCES public.users(id)

    );

CREATE TABLE ai.kb_articles ( id bigint NOT NULL DEFAULT

nextval('ai.kb_articles_id_seq'::regclass), domain character varying NOT

NULL,

        title text,

        text text,

        embedding_json jsonb, created_at timestamp without time zone DEFAULT

CURRENT_TIMESTAMP,

        CONSTRAINT kb_articles_pkey PRIMARY KEY (id)

    );

```

```

CREATE TABLE ai.practice_sessions ( id bigint NOT NULL DEFAULT
nextval('ai.practice_sessions_id_seq'::regclass), user_id bigint NOT NULL,
grade integer CHECK (grade >= 0 AND grade <= 100), major character varying,
created_at timestamp without time zone DEFAULT CURRENT_TIMESTAMP,
updated_at timestamp without time zone DEFAULT CURRENT_TIMESTAMP,
behavioral_feedback          text,
technical_feedback text,

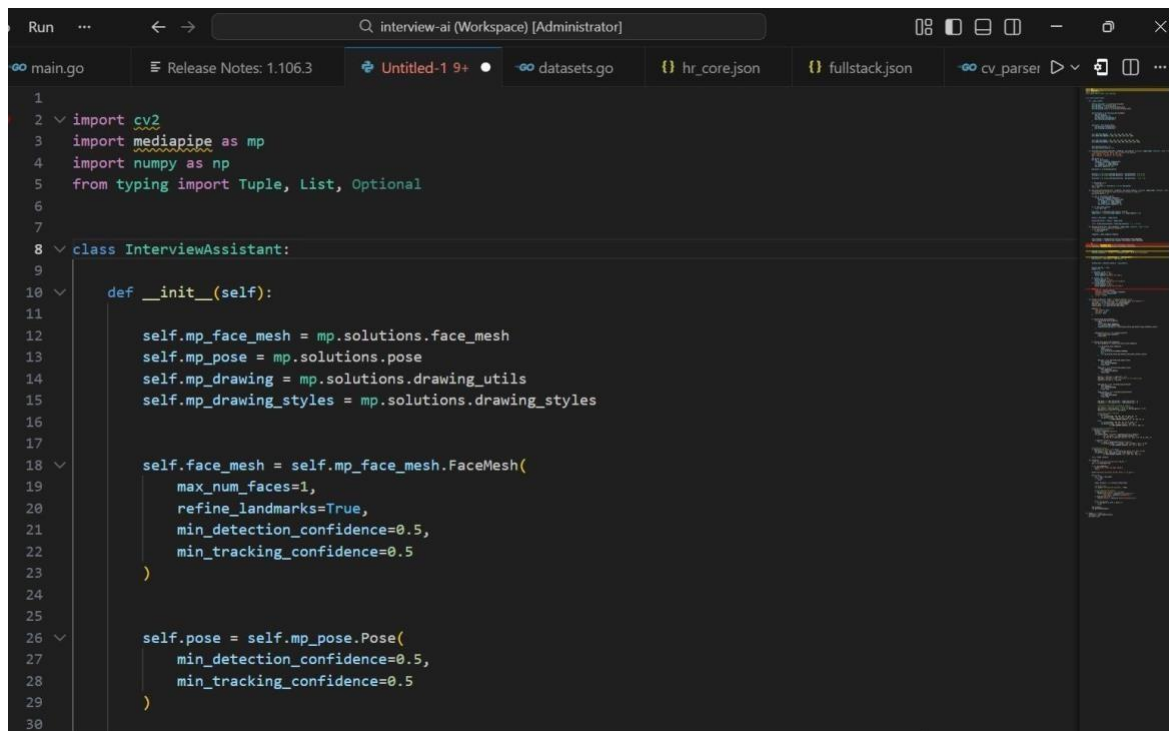
CONSTRAINT practice_sessions_pkey PRIMARY KEY (id),

CONSTRAINT practice_sessions_user_id_fkey FOREIGN KEY (user_id)
REFERENCES public.users(id)

);

```

APPENDIX B : Machine Learning Code



```

1
2  import cv2
3  import mediapipe as mp
4  import numpy as np
5  from typing import Tuple, List, Optional
6
7
8  class InterviewAssistant:
9
10     def __init__(self):
11
12         self.mp_face_mesh = mp.solutions.face_mesh
13         self.mp_pose = mp.solutions.pose
14         self.mp_drawing = mp.solutions.drawing_utils
15         self.mp_drawing_styles = mp.solutions.drawing_styles
16
17
18         self.face_mesh = self.mp_face_mesh.FaceMesh(
19             max_num_faces=1,
20             refine_landmarks=True,
21             min_detection_confidence=0.5,
22             min_tracking_confidence=0.5
23         )
24
25
26         self.pose = self.mp_pose.Pose(
27             min_detection_confidence=0.5,
28             min_tracking_confidence=0.5
29         )
30

```

```
Go Run ... < -> interview-ai (Workspace) [Administrator]
main.go 3 Release Notes: 1.106.3 Untitled-1 9+ datasets.go hr_core.json fullstack.json cv_parser

8 class InterviewAssistant:
10     def __init__(self):
11
12         self.pose = self.mp_pose.Pose(
13             min_detection_confidence=0.5,
14             min_tracking_confidence=0.5
15         )
16
17         self.LEFT_EYE_INDICES = [33, 133, 159, 145, 158, 153]
18         self.RIGHT_EYE_INDICES = [362, 263, 386, 374, 387, 380]
19
20         self.LEFT_EYE_CENTER = [33, 133, 157, 158, 159, 160, 161, 246]
21         self.RIGHT_EYE_CENTER = [362, 263, 388, 387, 386, 385, 384, 398]
22
23         self.posture_history = []
24         self.eye_contact_history = []
25
26     def calculate_eye_aspect_ratio(self, landmarks, eye_indices: List[int], image_shape: Tuple[int, int]) -> float:
27         """Calculate Eye Aspect Ratio (EAR) to detect eye openness.
28         Uses 6 points: [p1, p2, p3, p4, p5, p6]
29         EAR = (|p2-p6| + |p3-p5|) / (2 * |p1-p4|)
30         """
31         eye_points = []
32         for idx in eye_indices:
33             landmark = landmarks.landmark[idx]
34             x = landmark.x * image_shape[1]
35             y = landmark.y * image_shape[0]
36             eye_points.append([x, y])
37
38         eye_points = np.array(eye_points)
39
40         vertical_1 = np.linalg.norm(eye_points[1] - eye_points[5]) # p2 to p6
41         vertical_2 = np.linalg.norm(eye_points[2] - eye_points[4]) # p3 to p5
42
43         horizontal = np.linalg.norm(eye_points[0] - eye_points[3]) # p1 to p4
44
45         if horizontal == 0:
46             return 0.0
47         ear = (vertical_1 + vertical_2) / (2.0 * horizontal)
48         return ear
49
50     def calculate_eye_direction(self, landmarks, eye_center_indices: List[int], image_shape: Tuple[int, int]) -> Tu
51         """Calculate eye direction (gaze direction) relative to camera."""
52         eye_center_points = []
53
54         for idx in eye_center_indices:
55             if idx < len(landmarks.landmark):
56                 landmark = landmarks.landmark[idx]
```

```
main.go 3 Release Notes: 1.106.3 Untitled-1 9+ datasets.go hr_core.json fullstack.json cv_parser

43     def calculate_eye_aspect_ratio(self, landmarks, eye_indices: List[int], image_shape: Tuple[int, int]) -> float:
44
45         eye_points = []
46         for idx in eye_indices:
47             landmark = landmarks.landmark[idx]
48             x = landmark.x * image_shape[1]
49             y = landmark.y * image_shape[0]
50             eye_points.append([x, y])
51
52         eye_points = np.array(eye_points)
53
54         vertical_1 = np.linalg.norm(eye_points[1] - eye_points[5]) # p2 to p6
55         vertical_2 = np.linalg.norm(eye_points[2] - eye_points[4]) # p3 to p5
56
57         horizontal = np.linalg.norm(eye_points[0] - eye_points[3]) # p1 to p4
58
59         if horizontal == 0:
60             return 0.0
61         ear = (vertical_1 + vertical_2) / (2.0 * horizontal)
62         return ear
63
64     def calculate_eye_direction(self, landmarks, eye_center_indices: List[int], image_shape: Tuple[int, int]) -> Tu
65         """Calculate eye direction (gaze direction) relative to camera."""
66         eye_center_points = []
67
68         for idx in eye_center_indices:
69             if idx < len(landmarks.landmark):
70                 landmark = landmarks.landmark[idx]
```



```

8 class InterviewAssistant:
9     def analyze_posture(self, pose_landmarks, image_shape: Tuple[int, int]) -> dict:
10
11         landmarks = pose_landmarks.landmark
12
13         left_shoulder = landmarks[self.mp_pose.PoseLandmark.LEFT_SHOULDER]
14         right_shoulder = landmarks[self.mp_pose.PoseLandmark.RIGHT_SHOULDER]
15
16
17         left_hip = landmarks[self.mp_pose.PoseLandmark.LEFT_HIP]
18         right_hip = landmarks[self.mp_pose.PoseLandmark.RIGHT_HIP]
19
20         shoulder_y_diff = abs(left_shoulder.y - right_shoulder.y)
21         shoulder_alignment = "aligned" if shoulder_y_diff < 0.05 else "misaligned"
22
23         shoulder_center_y = (left_shoulder.y + right_shoulder.y) / 2
24         hip_center_y = (left_hip.y + right_hip.y) / 2
25
26         forward_lean = shoulder_center_y - hip_center_y
27
28         posture_quality = "good"
29         issues = []
30
31         if shoulder_y_diff > 0.1:
32             posture_quality = "poor"
33             issues.append("Shoulders not level")

```

```

def analyze_posture(self, pose_landmarks, image_shape: Tuple[int, int]) -> dict:

    if shoulder_y_diff > 0.1:
        posture_quality = "poor"
        issues.append("Shoulders not level")

    if forward_lean > 0.15:
        posture_quality = "poor"
        issues.append("Leaning too far forward")
    elif forward_lean < -0.1:
        posture_quality = "poor"
        issues.append("Leaning too far back")

    return {
        "quality": posture_quality,
        "shoulder_alignment": shoulder_alignment,
        "forward_lean": forward_lean,
        "issues": issues
    }

def process_frame(self, frame) -> Tuple[np.ndarray, dict]:
    rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    results_face = self.face_mesh.process(rgb_frame)
    results_pose = self.pose.process(rgb_frame)

    analysis = {
        "eye_contact": None,
        "eye_open": None,
        "posture": None
    }

```

```

9 def process_frame(self, frame) -> Tuple[np.ndarray, dict]:
10
11     analysis = {
12         "eye_contact": None,
13         "eye_open": None,
14         "posture": None
15     }
16
17     if results_pose.pose_landmarks:
18         self.mp_drawing.draw_landmarks(
19             frame,
20             results_pose.pose_landmarks,
21             self.mp_pose.POSE_CONNECTIONS,
22             landmark_drawing_spec=self.mp_drawing_styles.get_default_pose_landmarks_style()
23         )
24
25         analysis["posture"] = self.analyze_posture(
26             results_pose.pose_landmarks,
27             frame.shape
28         )
29
30     if results_face.multi_face_landmarks:
31         for face_landmarks in results_face.multi_face_landmarks:
32
33             self.mp_drawing.draw_landmarks(
34                 frame,
35                 face_landmarks,
36                 self.mp_face_mesh.FACEMESH_CONTOURS,
37                 None,
38                 self.mp_drawing_styles.get_default_face_mesh_contours_style()
39             )
40
41             left_eye = self.calculate_eye_aspect_ratio(
42                 face_landmarks,
43                 self.LEFT_EYE_INDICES,
44                 frame.shape
45             )
46
47             right_eye = self.calculate_eye_aspect_ratio(
48                 face_landmarks,
49                 self.RIGHT_EYE_INDICES,
50                 frame.shape
51             )
52
53             avg_eye = (left_eye + right_eye) / 2.0
54             eye_open = avg_eye > 0.25 # Threshold for eye open/closed
55             analysis["eye_open"] = eye_open
56
57     return frame, analysis

```

Ln 139, Col 63 Spaces: 4 UTF-8 CRLF Python 3.13.1 64-bit Go Live

```

9 def process_frame(self, frame) -> Tuple[np.ndarray, dict]:
10
11     if results_face.multi_face_landmarks:
12         for face_landmarks in results_face.multi_face_landmarks:
13
14             self.mp_drawing.draw_landmarks(
15                 frame,
16                 face_landmarks,
17                 self.mp_face_mesh.FACEMESH_CONTOURS,
18                 None,
19                 self.mp_drawing_styles.get_default_face_mesh_contours_style()
20             )
21
22             left_eye = self.calculate_eye_aspect_ratio(
23                 face_landmarks,
24                 self.LEFT_EYE_INDICES,
25                 frame.shape
26             )
27
28             right_eye = self.calculate_eye_aspect_ratio(
29                 face_landmarks,
30                 self.RIGHT_EYE_INDICES,
31                 frame.shape
32             )
33
34             avg_eye = (left_eye + right_eye) / 2.0
35             eye_open = avg_eye > 0.25 # Threshold for eye open/closed
36             analysis["eye_open"] = eye_open
37
38     return frame, analysis

```

```

8 class InterviewAssistant:
9     def process_frame(self, frame) -> Tuple[np.ndarray, dict]:
10
11         avg_ear = (left_ear + right_ear) / 2.0
12         eye_open = avg_ear > 0.25 # Threshold for eye open/closed
13         analysis["eye_open"] = eye_open
14
15         left_eye_dir = self.calculate_eye_direction(
16             face_landmarks,
17             self.LEFT_EYE_CENTER,
18             frame.shape
19         )
20         right_eye_dir = self.calculate_eye_direction(
21             face_landmarks,
22             self.RIGHT_EYE_CENTER,
23             frame.shape
24         )
25
26         avg_gaze_x = (left_eye_dir[0] + right_eye_dir[0]) / 2
27         avg_gaze_y = (left_eye_dir[1] + right_eye_dir[1]) / 2
28
29         eye_contact = abs(avg_gaze_x) < 0.15 and abs(avg_gaze_y) < 0.15
30         analysis["eye_contact"] = eye_contact
31
32         if eye_contact:
33             cv2.circle(frame, (50, 50), 20, (0, 255, 0), -1)
34             cv2.putText(frame, "Eye Contact", (80, 55),
35                 cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
36         else:

```

```

8 class InterviewAssistant:
139 def process_frame(self, frame) -> Tuple[np.ndarray, dict]:
140
141     if eye_contact:
142         cv2.circle(frame, (50, 50), 20, (0, 255, 0), -1)
143         cv2.putText(frame, "Eye Contact", (80, 55),
144             cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
145     else:
146         cv2.circle(frame, (50, 50), 20, (0, 0, 255), -1)
147         cv2.putText(frame, "No Eye Contact", (80, 55),
148             cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 0, 255), 2)
149
150     if analysis["posture"]:
151         posture = analysis["posture"]
152         y_offset = 100
153         cv2.putText(frame, f"Posture: {posture['quality'].upper()}",
154             (10, y_offset), cv2.FONT_HERSHEY_SIMPLEX, 0.7,
155             (0, 255, 0) if posture['quality'] == 'good' else (0, 0, 255), 2)
156
157         if posture['issues']:
158             for i, issue in enumerate(posture['issues']):
159                 cv2.putText(frame, f"- {issue}", (10, y_offset + 30 + i * 25),
160                     cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 255), 1)
161
162     if analysis["eye_open"] is not None:
163         eye_status = "Eyes Open" if analysis["eye_open"] else "Eyes Closed"
164         cv2.putText(frame, eye_status, (10, frame.shape[0] - 30),
165             cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 255, 255), 2)
166

```

```
def process_frame(self, frame) -> Tuple[np.ndarray, dict]:
    cv2.putText(frame, eye_status, (10, frame.shape[0] - 50),
                cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 255, 255), 2)
```

```
    return frame, analysis
```

```
def run(self):
```

```
    """Run the interview assistant with webcam."""
```

```
    cap = cv2.VideoCapture(0)
```

```
    if not cap.isOpened():
```

```
        print("Error: Could not open webcam")
```

```
        return
```

```
    print("Interview Assistant started. Press 'q' to quit.")
```

```
    while True:
```

```
        ret, frame = cap.read()
```

```
        if not ret:
```

```
            break
```

```
        frame, analysis = self.process_frame(frame)
```

```
        cv2.imshow('Interview AI Assistant', frame)
```

```
        if analysis["eye_contact"] is not None:
```

```
            print(f"Eye Contact: {analysis['eye_contact']}, "
```

```
                  f"Eyes Open: {analysis['eye_open']}")
```

```
def run(self):
```

```
    frame, analysis = self.process_frame(frame)
```

```
    cv2.imshow('Interview AI Assistant', frame)
```

```
    if analysis["eye_contact"] is not None:
```

```
        print(f"Eye Contact: {analysis['eye_contact']}, "
```

```
              f"Eyes Open: {analysis['eye_open']}")
```

```
    if analysis["posture"]:
```

```
        print(f"Posture: {analysis['posture']['quality']}")
```

```
    if cv2.waitKey(1) & 0xFF == ord('q'):
```

```
        break
```

```
    cap.release()
```

```
    cv2.destroyAllWindows()
```

```
if __name__ == "__main__":
```

```
    assistant = InterviewAssistant()
```

```
    assistant.run()
```